



University of Milano-Bicocca

**Department of Informatics, Systems and
Communication**

Master's Degree in Data Science

Auto Machine Learning for Financial Crisis Prediction

Supervisor: Prof. Mirko Cesarini

Co-supervisor: Prof. Fulvia Pennoni

Second Co-supervisor: Dr. Luca Bursa

Master's Degree Thesis by:

Induni Sandapiumi Nawarathna Pitiyage

Student ID Number 906451

Academic Year 2025/2026

To my parents: thank you for everything.

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Prof. Mirko Cesarini, for his invaluable guidance, constant encouragement, and for providing the academic freedom that allowed this research to flourish. His expertise and insights were fundamental to the completion of this work.

I am also deeply grateful to my co-supervisor, Prof. Fulvia Pennoni, and my second co-supervisor, Dr. Luca Bursa. Their technical support, meticulous feedback, and willingness to share their knowledge significantly enriched this thesis and my own academic growth.

Beyond the academic sphere, I owe a debt of gratitude to my family and cousins: Shani and Rasini. Their unwavering belief in my abilities and their constant emotional support provided the foundation I needed to persevere through the most challenging moments of this journey.

A special thank you goes to my friends, who made this experience truly memorable. In particular, I want to thank Sara and Vishmi, whom I was fortunate enough to meet during this journey. Their friendship, collaborative spirit, and help were instrumental in achieving this goal; I am grateful for the late-night study sessions and the encouragement that kept me moving forward.

Finally, thank you to everyone who contributed, directly or indirectly, to the successful completion of this Master's degree.

Abstract

Financial crises represent significant risks, yet traditional Early Warning Systems (EWS) often suffer from manual model selection bias and an inability to capture complex, non-linear interactions between macroeconomic variables. This thesis investigates the application of Automated Machine Learning (AutoML), specifically the tree-based Pipeline Optimization Tool (TPOT), to predict financial crises of countries from 1983 - 2017.

A central objective of this study is the construction of a balanced and representative dataset that accounts for the disparate financial and economic characteristics of different nations. To overcome the skewed data, the research introduces a metadata-driven stratified sampling strategy. This approach incorporates country-level information, including continent, land area, and GDP per capita, combined with binning techniques. By addressing the inherent class imbalance between the crisis and non-crisis observations, the study ensures that the model treats classes fairly.

The technical framework leverages rolling window feature engineering techniques to capture the temporal dynamics and lead-lag relationships inherent in economic cycles. Along with the standard variables, the enriched feature set serves as the input for TPOT, which utilizes Genetic Programming to automate selection of high-performing model architectures and hyperparameter configurations. To validate the efficacy of this automated approach, the study further compares the TPOT-evolved solutions against several manually constructed machine learning pipelines, architecturally inspired by AutoML outputs but optimized using stacking estimators to ensemble diverse base learners.

The results reveal important methodological insights: first, the incorporation of rolling window feature engineering significantly enhanced the model to capture the pre-crisis volatility, and temporal signals preceding historical episodes. Second, while the automated pipelines achieved superior internal validation scores, they exhibited a tendency toward overfitting the specific training architectures. This suggests that in the context of a noisy financial dataset, high-intensity optimization via AutoML may come at the expense of out-of-sample generalizability. However, challenges regarding data quality, missing values in emerging markets,

and the evolving nature of global economic patterns suggest that while EWS is highly effective, it requires continuous recalibration. While AutoML significantly enhances model discovery and reduces manual tuning effort, its black-box nature raises interpretability concerns that must be addressed for policy adoption. Overall, the findings demonstrate that evolutionary AutoML can serve as a powerful tool for financial crisis prediction when combined with tailored data preprocessing techniques, offering a promising foundation for future research on data-driven macro-financial surveillance systems.

Contents

List of Figures	viii
List of Tables	x
1 Introduction	1
1.1 Background and Motivation	1
1.2 Objectives of the Study	3
1.3 Scope of the Study	4
1.4 State of the Art	5
1.4.1 Existing Approaches	5
1.4.1.1 Machine Learning approaches to crisis prediction	7
1.4.2 Introduction to AutoML and TPOT	7
1.4.2.1 Why is Automation needed?	8
1.4.2.2 Traditional Machine Learning and AutoML	8
1.4.2.3 Overview of TPOT	9
1.4.2.4 Related Work on AutoML and TPOT algorithm	12
1.4.3 Research Gaps	13
1.5 Thesis Structure	14
2 Data and Methodology	15
2.1 Dataset Description and Sources	15
2.1.1 Exploratory Data Analysis	17
2.1.2 Data Preprocessing	20

2.1.3	Model Training	22
2.2	Research Design	24
2.2.1	Using TPOT for AutoML	25
2.2.2	Machine Learning Models	27
2.2.2.1	Multiple linear regression model	29
2.2.2.2	Probabilistic Models	32
2.2.2.3	Tree-based Models	33
2.2.2.4	Boosting models	35
2.2.2.5	Ensemble Methods	37
2.2.3	Evaluation Metrics	41
3	Illustrative Application	44
3.1	TPOT-Optimized Models	45
3.1.1	Comparative Analysis	45
3.1.2	Classification Performance and Confusion Matrix	46
3.1.2.1	Predictive Reliability	47
3.2	Comparative Framework of the Results of the AutoML Pipeline	48
3.2.1	Analysis of the Standard Dataset	48
3.2.2	The impact of Feature Engineering	50
4	Discussion	54
5	Conclusion	57
	Bibliography	61

List of Figures

1.1	<i>An illustration of the standard procedure for supervised machine learning. The grey area shows the part of machine learning process that can be automated by TPOT [50].</i>	10
2.1	<i>Frequency of the banking crisis around the world during the period 1983-2017</i>	19
2.2	<i>Distribution of countries with respect to each continent that experienced at least one banking crisis between 1983 and 2017.</i>	21
2.3	<i>The general workflow of TPOT pipeline. As the data set passes through the pipeline operators, the features are added, deleted, or altered. Finally, the combined dataset is fed to the final classifier to perform the final classification [50].</i>	25
2.4	<i>Workflow of TPOT Pipeline Optimization in the proposed method.</i>	26
2.5	<i>An example of the separable problem in a two dimensional space. The support vectors are outlined with black circle on the margin. The optimal hyperplane maximizes the separation between two classes [43].</i>	29
2.6	<i>Comparison of optimization trajectories for Stochastic Gradient Descent (left) and Batch Gradient Descent (right). The contour lines represent level sets of the objective (loss) function. The red paths illustrate the sequence of parameter updates toward the optimum. Stochastic Gradient Descent follows a noisy, zig-zag path due to gradient estimation from mini-batches, whereas Batch Gradient Descent moves smoothly along the steepest descent direction toward the minimum [53]</i>	31
2.7	<i>General Framework of (a) sequential ensemble and (b) parallel ensemble</i>	38

3.1	<i>The F1-score performance trends for four ML pipelines as a function of sample dimension. Each subplot corresponds to a different pipeline configuration: (a) Pipeline 1, (b) Pipeline 2, (c) Pipeline 3, and (d) Pipeline 5. In every panel, the blue curve represents training F1 scores while the red curve shows testing F1 scores, allowing direct comparison of model fit and generalization.</i>	50
3.2	<i>The F1-score performance trends for four ML pipelines as a function of sample dimension. Each subplot corresponds to a different pipeline configuration: (a) Pipeline 13, (b) Pipeline 14, (c) Pipeline 15, and (d) Pipeline 7. In every panel, the blue curve represents training F1 scores while the red curve shows testing F1 scores, allowing direct comparison of model fit and generalization.</i>	52

List of Tables

2.1	<i>Descriptive statistics for the full panel dataset covering 129 countries over the period 1983–2017. The table reports the mean, median, standard deviation, minimum, and maximum values for the dependent variable (Crisis) and the set of macroeconomic and financial explanatory variables. . .</i>	17
2.2	<i>Descriptive statistics of the explanatory variables disaggregated by crisis status (Crisis=1; Non-crisis=0) over the period 1983–2017. In contrast to Table 2.1, which reports pooled summary statistics for the entire sample, this table separates observations according to the occurrence of crisis events, thereby allowing a direct comparison of macroeconomic and financial conditions across regimes.</i>	18
3.1	<i>The experimental results from three distinct AutoML searches conducted through the TPOT framework. Each experiment shared a common goal of predicting the "crisis" target variable, while they significantly differed in data complexity and feature engineering strategies.</i>	45
3.2	<i>Classification performance of the best performed model (Experiment 3) conducted through the TPOT AutoML framework.</i>	47
3.3	<i>Confusion matrix analysis for the best performed model (Experiment 3) conducted through the TPOT AutoML framework.</i>	47
3.4	<i>Summary of the standard data with the evaluated model architectures and ensemble structures.</i>	49
3.5	<i>Summary of the Feature Engineered dataset with the evaluated model architectures and ensemble structures.</i>	51

4.1	<i>Comparison of the top-performing pipelines identified in Section 3: the standard dataset, the feature-engineered dataset, and the final TPOT optimization search. For each setting, the best pipeline is reported along with its model architecture and key evaluation metrics, including Class 1 precision, Class 1 recall, macro-F1, and overall accuracy. . . .</i>	55
-----	---	----

Chapter 1

Introduction

This chapter establishes the foundational framework for utilizing Automated Machine Learning (AutoML) as a robust Early Warning System (EWS) for financial crisis classification. Section 1.1 discusses the consequences of the financial instability and identifies the limitations inherent in traditional econometric approaches, along with the nature and the complexity of the data, advocating for the integration of the Tree-based Pipeline Optimization Tool (TPOT). Section 1.2 defines the primary research goals, specifically the development of an automated pipeline including preprocessing the data covering macroeconomic indicators to apply and compare multiple machine learning algorithms for crisis classification founded via TPOT autoML. Section 1.3 defines the boundaries of the study, such as the scope and challenges like data quality and model interpretability. Section 1.4 reviews the state-of-the-art of crisis prediction, outlining the statistical and ML approaches to crisis prediction and focusing on the AutoML and TPOT algorithm, including its strengths and limitations. Finally, Section 1.5 provides an overview of the organization of the thesis, summarizing the content and purpose of each subsequent chapter.

1.1 Background and Motivation

A financial crisis is a systemic banking crisis that is commonly defined as a situation in which the banking sector of a country experiences major bank runs, losses, or liquidations, coupled with the need for extensive government intervention in response to these losses as documented by Laeven and Valencia (2018) [41]. Such crises have severe global consequences, including deep and prolonged recessions, sharp contraction in credit and trade, rising unemployment, an increase in public debts due to bank bailouts, and a sharp crash in the value of the national currency which also leads to eliminating the government's ability to fund essential

services or respond to future shocks.

Consequently, the development of accurate classification models is of fundamental importance. Building an Early Warning System (EWS) in the context of finance is a diagnostic framework designed to identify economic and financial vulnerabilities that precede a systemic crisis. At its core, EWS analyzes economic indicators such as credit growth, inflation, and debt levels and determines when these variables deviate from their "normal" historical trends. When an indicator crosses a specific critical threshold, the system suggests that the probability of a crisis has significantly increased. Reliable early warning and crisis analysis models enable policymakers and international institutions to identify vulnerabilities in advance, implement preventative policies, and mitigate the severity and duration of crises. However, the growing complexity of the interconnectedness of the financial system makes crisis analysis challenging. Furthermore, macroeconomic associations are frequently non-linear, time-dependent, and influenced by country-specific dynamics, which limits the effectiveness of traditional econometric approaches.

Historically, crisis analysis and EWSs have relied on traditional statistical models such as logit and probit models. However, these models often impose strong assumptions about the linearity and variable interactions. As a result, they may struggle to capture complex, nonlinear patterns typically present in the financial system. Conversely, machine learning models are capable of handling nonlinearities, high-dimensional data, and complex interactions better than traditional econometric models. Although these models offer flexibility, designing an optimal ML model manually is time-consuming and requires deep expertise to navigate a vast search space of preprocessing techniques.

The empirical foundation of this research is based on the analysis of a comprehensive, unbalanced panel dataset encompassing 129 countries over 34 years (1983 - 2017)(for more details, see Pigini et al. (2017)). This dataset captures crisis events encoded as binary, taking the value 1 if a systemic banking crisis occurs in a given year or 0, along with a diverse array of economic indicators such as real GDP growth and inflation, monetary and financial variables like private credit expansion and net foreign assets. The complexity of this data, characterized by high-dimensional features and the inherent imbalance between crisis and non-crisis years, presents a significant challenge for traditional econometric modeling.

These challenges motivate the use of automated Machine Learning (AutoML) tools such as the Tree-Based Pipeline Optimization Tool (TPOT). TPOT relies on genetic programming to automate model selection, hyperparameter tuning, preprocessing, and feature engineering. This process reduces human bias; in other words, human researchers often rely on familiar preprocessing techniques, overlooking complex combinations. This approach replaces manual testing of different combinations with a

genetic algorithm, allowing for efficient exploration of large hyperparameter spaces where optimal pipeline configurations are unlikely to emerge through traditional, manually designed modeling procedures. Implementing custom preprocessing steps to address specific data challenges, such as class imbalance, before running through the automated pipeline search further enhances the potential for improving classification performance.

Despite the growing interest in machine learning for crisis analysis, the use of AutoML in this domain, particularly in the presence of severe imbalance, remains limited. To address this gap, this study aims to evaluate whether automated pipeline optimization can outperform standard machine learning approaches in classifying and predicting financial crises.

1.2 Objectives of the Study

The primary objective of this study is to investigate the predictive performance and operational efficiency of Automated Machine Learning (AutoML) in the classification of financial crises. Specifically, the research employs the TPOT framework to autonomously identify high-performing machine learning pipelines, moving beyond traditional manual model selection approaches. To achieve this objective, the study relies on an unbalanced panel dataset spanning more than three decades (1983–2017) and covering 129 countries, enabling a comprehensive long-run evaluation of early warning systems (EWSs) for financial crises.

A central objective is to construct a balanced and representative dataset that ensures fair representation of countries with heterogeneous financial and economic characteristics. To this end, the research introduces a metadata-driven stratified sampling strategy that incorporates country-level information such as continent, land area, and GDP per capita, combined with binning techniques to address the inherent imbalance in crisis and non-crisis observations. This approach aims to mitigate bias arising from structural heterogeneity across countries while preserving the underlying economic diversity of the sample.

Another key objective is to assess whether the inclusion of temporal dynamics through a rolling-window framework enhances the robustness and stability of EWSs compared to static modeling approaches based solely on contemporaneous covariates. In line with this goal, model performance is optimized with respect to the Macro F1-score, ensuring balanced predictive accuracy between crisis and non-crisis events in the presence of class imbalance.

Furthermore, the study seeks to systematically evaluate and document the performance of high-complexity machine learning pipelines discov-

ered via AutoML. While TPOT identifies candidate pipelines, this research extends its functionality by developing custom components that enable reproducible parameter control, probability extraction, and automated pipeline evaluation with standardized reporting. The resulting models ranging from simple classifiers to multi-layered stacking architectures are assessed using standardized performance metrics, including accuracy, precision, recall, F1-score, and learning curves. Ultimately, this framework contributes to transforming inherently “black-box” AutoML outputs into transparent, interpretable, and empirically validated early-warning tools for financial crisis prediction.

1.3 Scope of the Study

The scope of the study is centered on the application of the TPOT AutoML pipeline to a dataset of macroeconomic indicators stored in a SQLite environment. Geographically and temporally, the original dataset encompassed missing data. To address this, custom preprocessing steps such as handcrafted lagged features or feature transformations were incorporated into the dataset, prior to being analyzed with AutoML. The analysis leverages a set of core covariates, including real GDP growth, inflation rate, foreign exchange ratios, and domestic credit growth, which are leading indicators of financial instability. A primary distinction in this study is the comparison of two data structures: a standard dataset with the inclusion of the covariates present in the original dataset and a rolling-window dataset that incorporates lagged crisis observations. Furthermore, to represent a balanced dataset with a fair representation of the countries with different financial backgrounds, a meta-dataset was created through web scraping techniques for each country to provide additional contextual information, including continent and land area.

To achieve these objectives, the study encompasses the following essential components:

1. Temporal and Geographic Coverage

- The analysis leverages historical macro-financial data from 1983-2017, encompassing a range of economic indicators and crisis labels on a multi-country framework, ensuring generalizability across different economic contexts and regions.

2. Computational framework

- The system is developed entirely using a combination of open-source tools to ensure reproducibility and cost-efficiency.

- Exploratory Phase (R): Data exploration and initial processing, including visualizations and preliminary statistical analysis, are conducted in R.
- Modeling Phase (Python): The core modeling pipeline is implemented in Python, adapting libraries such as matplotlib for performance plotting and scikit-learn for foundational tasks.
- AutoML (TPOT): The heart of the technical scope involves the Tree-based Pipeline Optimization Tool (TPOT). This process leverages genetic programming to automatically search over a vast space of models and preprocessing steps and models, including Decision Trees, Naive Bayes, Extra Trees, and XGBoost [50].
- Optimization Parameters: The search space is defined as 100 generations with a population size of 100, optimizing the hyperparameter and model selection based on cross-validation to ensure the most effective predictive pipeline.

1.4 State of the Art

This section presents the state-of-the-art in crisis prediction, outlining the current methodologies and knowledge related to this research. The Sub-section 1.4.1 reviews the existing approaches to crisis prediction, focusing on the strengths and the limitations of the traditional statistical approaches such as logit and probit models. The Sub-section 1.4.1.1 explores the growing application of machine learning techniques for crisis prediction focusing on their algorithm and feature engineering strategies. The Sub-section 1.4.2 focuses on the AutoML and TPOT algorithm, including automated machine learning, strengths, and limitations of TPOT mechanics. Finally, Sub-section 1.4.3 identifies research gaps, highlighting unresolved challenges, limitations in existing literature, and opportunities for developing more accurate and robust predictive frameworks.

1.4.1 Existing Approaches

EWS models are built to predict the systemic risk of the financial system. One of the earliest approaches to crisis prediction is the signal approach, a threshold-based indicator method proposed by Kaminsky and Reinhart [55]. The basic concept of this approach is that for each of the economic/-financial indicators, a threshold was imposed beyond which the indicator “flashes” a signal. One of the main drawbacks of this approach is that the thresholds are fixed and may not adapt well to changing economic regimes, and it ignores nonlinear interactions between indicators.

Along this line of work, classical econometric approaches, particularly logit models, have been widely applied in the literature to predict financial crises, yet they face challenges in panel data. One of the key methodological challenges arises from permanent country-specific unobserved heterogeneity, which could be accounted for by the inclusion of fixed effects. However, because crises are rare events, fixed-effects estimation is often avoided to prevent the loss of a sizable number of countries. This limitation is due to the complete separation problem: when a country never experiences a crisis, the maximum likelihood (ML) estimate of its intercept tends toward negative infinity, preventing its inclusion in the estimation sample. Simply retaining the whole sample while excluding the intercepts of such countries leads to biased ML estimates [29]. Moreover, in short panel time-series, the ML estimator of fixed-effects binary choice models is additionally biased due to the incidental parameters problem [48], [42].

To address the above issues, some studies have proposed logit-based EWS using a penalized maximum likelihood (PML) approach, applying the bias reduction technique introduced by Firth (1993) [19] and Kosmidis and Firth (2009) [40]. This method allows the inclusion of fixed effects while retaining the full sample of countries [29]. However, one of the major problems encountered in this study is that it cannot forecast "first-ever" crises, which means if a country has never experienced a crisis, the model has no fixed effect for it, so you can't produce meaningful forecasts for that first crisis event. Furthermore, estimating many fixed effects with short time series causes bias in other parameter estimates [52].

While binary logit models have been widely used in early applications, they are often subject to post-crisis bias because they do not differentiate between crisis phases. To overcome this, several studies have adopted the multinomial logit model [34], which explicitly distinguishes between tranquil, crisis, and post-crisis periods. For example, Bussière and Fratzscher [11] applied this method in a currency crisis context, while Oet *et al.* [49] used it for banking crises. In these applications, the dependent variable typically takes the value of 1 for the first crisis year, 2 for subsequent crisis years, and 0 for non-crisis years, with the probability of each state modeled as a function of multiple macroeconomic and financial indicators. This approach allows for richer phase-specific dynamics to be captured compared to the binomial specification.

Collectively, these studies established the foundation for empirical crisis prediction, but they also revealed key statistical limitations that have motivated the development of more robust and flexible methods in recent years.

1.4.1.1 Machine Learning approaches to crisis prediction

The use of machine learning algorithms for predicting financial crises has gained significant prominence, offering distinct advantages over the traditional regression-based approaches. First, ML models tend to achieve superior predictive performance in complex, high-dimensional settings. Secondly, ML models are flexible enough to capture complex data structures in the data, including nonlinear relationships and interactions. This flexibility is particularly helpful because financial crises are rare and extreme events that often exhibit nonlinear dependencies [1]. Previous studies employing conventional regression approaches (that typically require prior specification of the nonlinearities and interactions) emphasized the importance of focusing on the distribution and interactions of explanatory variables [5]; [36]; [36]. To this end, ML approaches can automatically and efficiently search across a much wider spectrum of nonlinearities and interactions without the need for pre-specification.

Empirical evidence supports this advantage: Alessi and Detken [1] and Tanaka, Kinkyo, and Hamori [63] find that random forests outperform both the logit model and traditional signaling approaches in early-warning settings. Holopainen and Sarlin [31] extend these findings to other ML methods such as artificial neural networks, support vector machines, k -nearest neighbors, and decision trees. Unlike logit models, many machine learning algorithms are highly parameterized, which allows them to capture complex relationships but also increases the risk of overfitting and poorer out-of-sample performance. Mitigating this risk requires careful hyperparameter tuning, often via cross-validation adapted to the panel and time-series structure of crisis data [45].

To address the interpretability or black-box predictions, recent work has incorporated model-agnostic explanation tools, including the Shapley value decompositions combined with regression frameworks [62]; [37]; [9]. This approach decomposes ML predictions into the contribution of individual variables, enabling the identification and statistical testing of the key economic drivers underlying the crisis forecast. Such techniques account for the explanatory variable transparency often required in economic policy contexts.

1.4.2 Introduction to AutoML and TPOT

This section provides a deep dive into automation in ML by addressing the main challenges and insights about the Tree-based Pipeline Optimization Tool (TPOT) AutoML.

1.4.2.1 Why is Automation needed?

The modern society is heavily reliant on automation which, over the centuries, has enabled us to eliminate humans from repetitive and hazardous tasks. Automation can be briefly described as a process of using technology or algorithms to perform tasks with minimal human intervention. In the context of data science and machine learning, automation aims to streamline repetitive and time-consuming steps such as data preprocessing, feature selection, model training, and hyperparameter tuning. Instead of relying on manual experimentation and expert intuition, automation enables systems to make adaptive decisions based on predefined logic, optimization criteria, or heuristic search methods.

The need for automation in machine learning arises from the increasing complexity of modern data-driven applications. Moreover, as the dataset grows larger and more heterogeneous, the expertise required to manage and tune machine learning pipelines becomes a limiting factor. Automation helps to overcome these challenges by offering the following key advantages:

- Time saving: Repetitive automation helps to overcome the time-consuming operations, thus saving time and computational resources.
- Accuracy and consistency: Humans are prone to errors and inconsistencies. Therefore, without substantial manual adjustment, automation tools may frequently build models that are competitive.
- User-friendly: Automation allows non-expert users to build robust predictive models while enabling experts to focus on strategic interpretation and domain-specific insights.

1.4.2.2 Traditional Machine Learning and AutoML

Many machine learning tools require professional knowledge of the methods and the problem to understand the underlying assumptions of the analysis and apply them effectively. For instance, Figure 1.1 illustrates the typical steps a data scientist would follow when approaching an ML problem that traditionally requires expertise in multiple areas, including data preprocessing, feature engineering, model selection, and hyperparameter optimization. In particular, the effectiveness of ML methods depends heavily on hyperparameter tuning. Poorly chosen hyperparameters can lead to underfitting or overfitting.

Traditional methods to hyperparameter tuning include manual adjustment, grid search, and random search [4]. While manual tuning can be time-consuming and produce inconsistent results, grid search, which is

currently the most common form of hyperparameter optimization, applies a systematic brute force search to identify the optimal parameter set; however, typically, it comes at the cost of significant computational resources. Random search, in contrast, samples combinations at random and has been shown to locate near-optimal configurations more efficiently than exhaustive search in many cases. A more advanced method is Bayesian optimization [8], a probabilistic model that outperforms manual expert tuning and brute-force methods, especially when model evaluations are expensive.

In recent years, AutoML has emerged as a promising solution to challenges in algorithm selection, hyperparameter tuning, and workflow design. As stated in Section 1.4.2.1, AutoML also helps to overcome challenges such as time consumption, accuracy, consistency across different machine learning workflows, and substantially reduces human error. The fundamental distinction between traditional ML and AutoML lies in the degree of human involvement and automation. In traditional ML, humans drive the entire process, making crucial design choices and manually tuning model parameters. AutoML, conversely, delegates these tasks to algorithms that systematically search the space of possible models and parameter settings. As a result, AutoML not only accelerates the model development cycle but also reduces the potential for subjective decision-making.

Furthermore, AutoML encompasses different frameworks, which include Pycaret, H2O, TPOT, Auto-sklearn, and FLAML, each offering unique features. AutoML automates the end-to-end process of applying machine learning to real-world problems, from data preprocessing to model training and evaluation.

1.4.2.3 Overview of TPOT

Several tools have been developed to operationalize the AutoML paradigm. Among them, the Tree-based Pipeline Optimization Tool (TPOT) is an open-source Python library designed to automate the process of building and optimizing ML pipelines. Many steps in the machine learning process (depicted in Figure 1.1), including feature engineering, model selection, and hyperparameter tweaking, are automated by AutoML tools and frameworks.

The TPOT library [3] leverages genetic programming to generate optimized analysis pipelines for predictive tasks [44]. TPOT is composed of a series of feature selectors, pre-processors, and ML models with the objective of maximizing classification and prediction accuracy, efficiency, and accessibility. While the majority of AutoML systems concentrate primarily on model selection, choosing the most suitable algorithm for a given task, and hyperparameter optimization, TPOT goes further by also

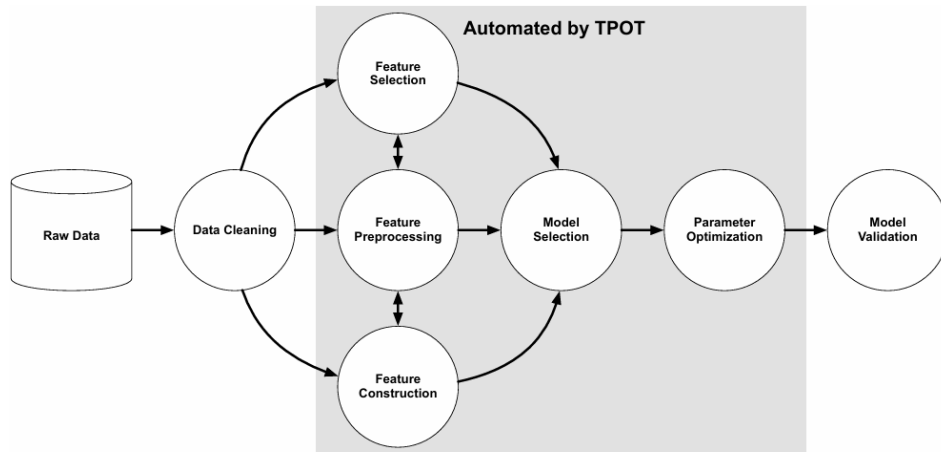


Figure 1.1: *An illustration of the standard procedure for supervised machine learning. The grey area shows the part of machine learning process that can be automated by TPOT [50].*

incorporating feature engineering and feature selection, which involves identifying the most relevant variables. It evaluates entire pipelines using cross-validated scores, such as balanced accuracy or mean squared error, without requiring prior knowledge of the problem. Due to these appealing properties, TPOT has been shown to frequently outperform standard ML analyses [50].

Overview of Genetic Programming

At the core of TPOT’s optimization process is genetic programming, a type of evolutionary algorithm inspired by biological evolution. In GP, optimization proceeds through several fundamental steps: initialization, evaluation, selection, and genetic variation. In TPOT, each pipeline is treated as an individual in a population, with its components, such as preprocessing steps and predictive models, represented as genes. The process begins with initialization, generating a population of random pipelines, which are then evaluated based on their performance using cross-validation metrics. During selection, the best-performing pipelines are chosen to reproduce, while genetic variation occurs through operations such as crossover (combining parts of two pipelines) and mutation (randomly altering a pipeline’s structure or parameters). Over multiple generations, TPOT evolves pipelines toward higher performance, gradually improving both predictive accuracy and structural efficiency. This biologically inspired search mechanism allows TPOT to explore a vast space of possible pipeline configurations more effectively than traditional random or grid search methods (see also Figure 2.4).

TPOT Limitations and Strategies to Overcome

TPOT has certain limitations, despite its benefits. One of the major disadvantages is its computational cost. TPOT is an evolutionary process that requires assessing many candidate pipelines, each involving various model training and validation iterations. As a result, the process is time-consuming, particularly for large datasets or complicated search spaces. To mitigate this issue, researchers can constrain the search space by limiting the number of generations or population size, or enable techniques such as early stopping, which halts the pipeline optimization once performance ceases to improve, reducing computation time and preventing overfitting.

Additionally, the reliance of TPOT on scikit-learn restricts its flexibility, as it primarily supports models and transformations available within that ecosystem [54]. For instance, deep learning models, time series forecasting frameworks, or domain feature extraction techniques are not naively integrated into scikit-learn components. Furthermore, each dataset requires its own preprocessing procedures prior to being analyzed with AutoML, as the optimal preprocessing steps depend on factors such as missing data types, variable types, and data quality. Therefore, when modeling a financial prediction task to ensure better performance, it is necessary to combine AutoML with custom preprocessing and post-processing steps, such as handcrafted lagged features or feature transformations. These techniques allow the user to retain the advantages of TPOT’s automation while still applying methods relevant to financial crisis forecasting. The stochastic nature of TPOT’s genetic programming algorithm introduces another limitation: different runs may produce different pipelines, and the algorithm does not guarantee that the global optimum will be found. This challenge can be reduced by setting a fixed random seed for reproducibility and performing several runs to increase the likelihood of converging to a reliable solution. Additionally, leveraging ensemble methods or model stacking techniques can help mitigate the risk of relying too heavily on a single algorithm [65]. Implementing effective hyperparameter tuning strategies, such as grid search, random search, or Bayesian optimization, can enhance the performance of AutoML-generated models.

Finally, interpretability may be reduced when complex pipelines are produced, making it more difficult to explain the underlying logic of the optimized model [65]. Prioritizing the transparency and the understandability of the complex workflows helps to drive impactful scientific discoveries. Therefore, maintaining thorough documentation of the AutoML pipeline, including the sequence of preprocessing steps, model choices, hyperparameter settings, and feature transformations, enhances interpretability by allowing researchers to trace and understand how predictions are generated.

Despite these limitations, TPOT has found wide applicability across many fields such as bioinformatics, finance, marketing, and environmental science, where researchers leverage its automation capabilities to identify effective and efficient predictive models without extensive manual tuning.

1.4.2.4 Related Work on AutoML and TPOT algorithm

Numerous research studies have examined AutoML’s influence on model performance and workflow efficiency, outlining both its benefits and drawbacks.

One of the first contributions to the field of AutoML emphasized automation of both feature engineering and pipeline selection. Deep Feature Synthesis [38] introduced automated feature engineering as a synthetic step in the data science pipeline, while later frameworks targeted with the introduction of Auto-sklearn by Feurer et al. [18], an extension of the scikit-learn library that automates end-to-end model selection and hyperparameter tuning. Comprehensive reviews and edited volumes [33] also frame AutoML as a suite of techniques for feature engineering, model search, hyperparameter tuning, and meta learning. One conclusion that can be drawn from these studies is that AutoML can reach competitive performance when compared to manually tuned models, despite requiring substantial computational resources.

To overcome the inability of Auto-sklearn to produce arbitrarily large pipelines for AutoML, genetic programming was introduced in 2015, [70], to optimize machine learning pipelines. Moreover, TPOT is an AutoML tool based on genetic programming that optimizes machine learning pipelines, was proposed by Olson et al., [50]. Furthermore, foundational studies [50], [51] demonstrate the ability of the TPOT to discover competitive pipelines without human intervention and further highlight the effectiveness of the evolutionary optimization of the pipeline on a given dataset. Several recent studies provide external validation of AutoML tools in diverse settings. Ribeiro, Orzechowski, Wagenaar, and Moore, [57] compared multiple AutoML algorithms and showed that performance depends on the underlying characteristics of the dataset. Another recent contribution extending TPOT’s capabilities by Le, Fu, and Moore, [44], which demonstrates that TPOT can be scaled to high-dimensional biomedical datasets by incorporating feature set selectors into the search process.

Typically, financial prediction tasks are composed of imbalanced classes and noisy indicators. Therefore, custom preprocessing procedures, such as robust feature engineering, are crucial. AutoML systems, particularly pipeline-oriented approaches like TPOT, reduce reliance on model building through trial and error and enable automated exploration of

preprocessing steps and feature transformation. However, as highlighted in Ribeiro et al. [57] and in He, Zhao, & Chu’s survey, [13], AutoML systems must be adapted to domain constraints and may require specialized validation schemes to avoid overfitting in temporal data. This study ([57]) shows that the performance of AutoML algorithms varies depending on the characteristics of the dataset. They specify that some datasets require specific preprocessing or strategies to prevent misleading results. In temporal domains such as financial crisis prediction, data points are not independent; their chronological order carries crucial predictive performance, and also the imbalanced nature of the dataset results in under-representation of the minority class. If an AutoML system randomly splits the data into training and testing sets, ignoring time order and the imbalanced nature, the model may learn from the majority class and might learn from future information during training, resulting in data leakage and overly optimistic performance estimates. Therefore, when applying AutoML to financial time series, it is necessary to adapt the validation strategy using time-aware methods such as rolling-window techniques to avoid overfitting and ensure that the model’s predictive power reflects real-world forecasting conditions.

1.4.3 Research Gaps

Despite the growing attention of literature on financial crisis prediction and EWSs, the practical application of Early Warning Systems (EWSs) by policymakers is limited, especially in international financial institutions, despite the substantial body of studies on their ability to foresee banking crises. There are several gaps in the application of modern ML approaches, particularly in the use of Automated Machine Learning (AutoML) frameworks such as TPOT. While machine learning models like Random Forests, Support Vector Machines, and Gradient Boosting have been widely explored in the prediction of crises [11]; [56]), only a limited number of studies ([69],[44]) have investigated the use of AutoML for systematically optimizing model pipelines in this domain. Given that TPOT can automate feature selection, preprocessing, and hyperparameter tuning, it enables data-driven model optimization while maintaining interpretability, reducing manual effort, and improving the reliability of predictive results [50].

Another significant limitation in previous research lies in data sampling strategies, which are crucial for ensuring a balanced dataset so that predictions are not biased toward particular instances. Many studies have relied on random sampling or simple train/test splits, which can lead to biased results when dealing with highly imbalanced crisis data [39]. Moreover, limited attention has been given to integrating country-specific metadata such as geographic location, land area, or GDP per capita into sampling procedures. Without this, stratification may fail to provide

a fair representation of countries with diverse financial and geographical characteristics, potentially biasing model predictions toward more frequently observed groups. Existing works rarely consider how heterogeneity in economic structure (e.g., small vs. large economies, high- vs. low-income countries) may influence crisis prediction outcomes.

In addition to addressing limitations in dataset coverage and sampling, we also advance the methodological framework by leveraging TPOT AutoML for automated pipeline optimization. While many studies manually select and tune a handful of classifiers, our approach integrates a pipeline search space that systematically evaluates and optimizes feature transformations, model hyperparameters, and ensemble strategies.

1.5 Thesis Structure

The remainder of this thesis is structured as follows:

- Chapter 2 describes the data sources, preprocessing steps, and the selection of variables used in the analysis, along with descriptive and exploratory data analysis. Furthermore, this chapter outlines the machine learning methodologies employed, including the TPOT model design for AutoML, training procedures, evaluation metrics, and feature importance analysis.
- Chapter 3 presents the empirical results, including model performance comparisons.
- Chapter 4 discusses the findings in the context of economic theory and policy implications.
- Chapter 5 concludes the study with a summary of key insights, contributions, and directions for future research.

Chapter 2

Data and Methodology

This chapter presents the insights into the dataset and methodological framework adapted in this study. Section 2.1 begins by describing the dataset and its sources, followed by an in-depth descriptive and exploratory data analysis to uncover initial patterns. The subsequent Section 2.1.2 details the data preprocessing steps undertaken to ensure the data quality and suitability for modeling, and Section 2.1.3 outlines the proposed approach for the train set and test set division. In the second part, Section 2.2 provides a deep dive into the implementation of the TPOT AutoML, how it is used to handle automation in machine learning, and further explores different machine learning algorithms systematically found via TPOT AutoML, followed by the evaluation metrics that are used to evaluate the proposed method.

2.1 Dataset Description and Sources

The proposed method is evaluated using an unbalanced panel dataset comprising 129 countries over 34 years (1983–2017). The initial dataset was considered in Pigni (2021) and includes annual macroeconomic and financial indicators for each country (see also *Table 1* for a summary).

Dependent Variable

The dependent variable in this study is the occurrence of a systemic banking crisis. This variable is defined according to Laeven and Valencia [41], who classify a banking crisis as an event in which a country experiences significant signs of financial distress in its banking system. Such distress may include bank runs, substantial losses in the banking system, bank liquidations, or large-scale government interventions aimed at rescuing financial institutions due to severe solvency problems. The crisis variable is binary, taking the value 1 if a systemic banking crisis occurs in a given year and 0 otherwise.

Explanatory Variables

The set of explanatory variables was compiled following the Early Warning System (EWS) literature, drawing on the contributions of Demirgüç-Kunt and Detragiache [17], Davis and Karim [16], and Caggiano et al. [12]. To account for the predictive nature of early-warning models, all explanatory variables are lagged by one period, ensuring that they reflect information available prior to the crisis event.

The explanatory variables are grouped into three main categories:

Macroeconomic Indicators

These variables capture the overall economic conditions of a country. With the exception of the real interest rate, they are generally expected to have a negative relationship with the probability of a systemic banking crisis.

- Real GDP growth rate
- Logarithm of GDP per capita
- Inflation rate
- Real interest rate

Monetary Variables

These indicators represent monetary and credit conditions within the financial system. Both are expected to increase the likelihood of a banking crisis, particularly when credit expansion leads to excessive leverage and deteriorating asset quality.

- Broad money (M2) to foreign exchange ratio
- Growth rate of private credit

Financial Variables

These variables describe external financial positions and international exposure.

- Growth rate of net foreign assets to GDP (expected to reduce the probability of a crisis).

2.1.1 Exploratory Data Analysis

To provide a comprehensive understanding of the dataset, descriptive statistics are first presented in a Table 2.1 for the entire sample covering all countries and years from 1983 to 2017. This overall summary offers an overview of the data structure, variable ranges, and dispersion, as well as an initial assessment of skewness and potential outliers.

Table 2.1: *Descriptive statistics for the full panel dataset covering 129 countries over the period 1983–2017. The table reports the mean, median, standard deviation, minimum, and maximum values for the dependent variable (Crisis) and the set of macroeconomic and financial explanatory variables.*

Variable	Mean	Median	SD	Min	Max
Dependent Variable					
Crisis	0.05	0.00	0.22	0.00	1.00
Explanatory Variables					
Real GDP growth rate	2.75	2.44	3.98	-13.82	18.18
Logarithm of GDP per capita	5.47	6.74	3.71	0.00	11.39
Inflation rate	18.81	2.98	273.85	-31.57	13,611.63
Real interest rate	1.69	0.00	9.52	-46.11	66.15
(M2) to foreign exchange ratio	10.80	2.59	29.12	0.00	145.00
Growth of private credit	11.53	6.91	28.65	-631.04	518.08
Growth of net foreign assets to GDP	0.01	0.00	0.19	-2.44	2.30

As depicted in the Table 2.1, the dependent variable, *Crisis* has a mean value of 0.05, reflecting that only about 5% of the observations correspond to crisis periods, suggesting the rarity of such events and the imbalanced nature of the dataset. Among the explanatory variables, the average of the *real GDP growth rate* is 2.75%, with a moderate variability across the countries and time, while *logarithm of GDP per capita* shows a mean of 5.47, suggesting considerable income heterogeneity across the economies of the countries. The *inflation rate* exhibits an extremely high dispersion, with a standard deviation of 273.85 and an extremely wide range from -31.57 to over 13,600, indicating the presence of extremely high outliers likely associated with hyperinflation episodes in certain countries. Similarly, *the growth of private credit*, *(M2) to the foreign exchange ratio*, and *the real interest rate* show substantial variability, reflecting differences in financial depth and monetary conditions.

However, given that the main objective of this research is to predict the occurrence of financial crisis, it is also crucial to examine whether the statistical properties of the variables differ between crisis and non-crisis observation. Therefore, in addition to global descriptive summary, the dataset is further divided into two subgroups, associated with crisis events (crisis = 1) and those countries without crisis (No-crisis = 0) (see

Table 2.2 for summary).

Table 2.2: *Descriptive statistics of the explanatory variables disaggregated by crisis status (Crisis=1; Non-crisis=0) over the period 1983–2017. In contrast to Table 2.1, which reports pooled summary statistics for the entire sample, this table separates observations according to the occurrence of crisis events, thereby allowing a direct comparison of macroeconomic and financial conditions across regimes.*

Variable	Crisis			Non-Crisis		
	Mean	Median	SD	Mean	Median	SD
Explanatory Variables						
Real GDP growth rate	1.41	1.86	5.39	2.83	2.47	3.88
Logarithm of GDP per capita	7.20	7.42	2.23	5.38	6.70	3.75
Inflation rate	78.26	9.09	539.23	15.62	2.71	251.55
Real interest rate	2.96	1.81	15.18	1.62	0.00	9.11
(M2) to foreign exchange ratio	22.77	4.25	42.78	10.15	2.46	28.07
Growth of private credit	21.67	14.38	48.95	10.99	6.58	27.04
Growth of net foreign assets to GDP	0.00	0.00	0.23	0.01	0.00	0.19

In general, the descriptive statistics shown in the Table 2.2 indicate the substantial difference in the economic conditions associated with the two states. During crisis periods, the average *real GDP growth rate* is notably lower (1.41) compared to non-crisis periods (2.83), suggesting weaker economic performance. Similarly, the mean *logarithm of GDP per capita* is higher during the crisis, reflecting that crises tend to occur more frequently in middle or high-income economies. *Inflation rate* exhibits a very high mean and standard deviation during the crisis, suggesting that a few hyperinflation episodes inflate the average. The median of *inflation rate* is far lower, confirming the distribution is highly skewed. The *real interest rate* also shows a great variability during crisis with a standard deviation of 15.18 compared to 9.11 in non-crisis times. Furthermore, *the ratio of broad money (M2) to foreign exchange* and the *growth of private credit* are higher and more volatile in crisis periods, implying potential financial instability and liquidity pressures. Finally, the *growth of net foreign assets to GDP* remains relatively stable across both groups, with minimal variation.

Exploratory Data Analysis

Figure 2.1 presents a choropleth map illustrating the countries included in the dataset and the global frequency of banking crises from 1983–2017. Among the countries analyzed, Nigeria, Philippines, and Ukraine recorded the highest frequency of banking crises, each experiencing nine crisis episodes. Uruguay followed with seven crises, while Costa Rica, Hungary, Mexico, Nicaragua, and the United States each experienced six. In contrast, large emerging economies such as India and China reported only one crisis event during the same period, indicating relatively greater

financial stability and institutional resilience despite their rapid economic growth.

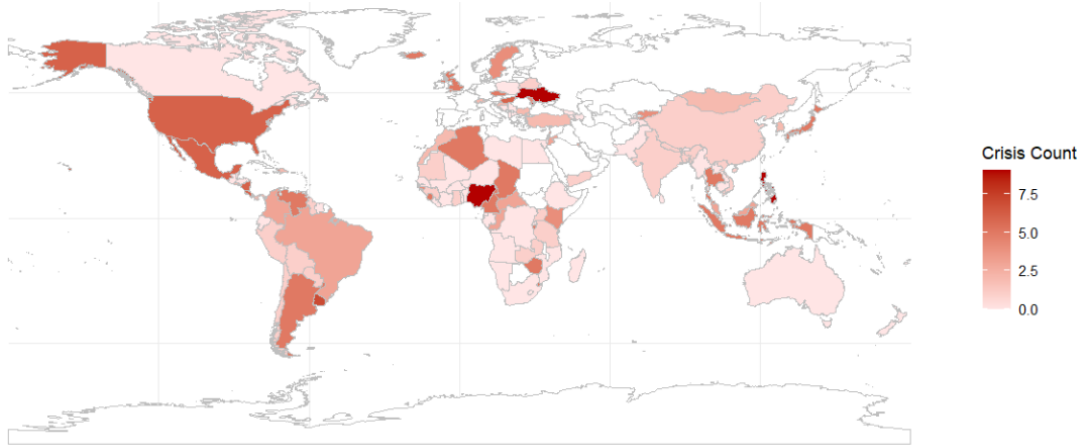


Figure 2.1: *Frequency of the banking crisis around the world during the period 1983-2017*

To further analyze the crisis events, Figure 2.2 shows the countries that experienced at least one banking crisis, and the countries with no crisis events are excluded from the visualization. The plot highlights the variation in crisis occurrence across countries and continents during the 1983-2017 period. To contextualize the spatial patterns depicted in Figure 2.2, it is important to recall the major banking crisis that shaped the global financial landscape during this period.

- Early 1980s: The Latin American Debt Crisis, marked one of the earliest financial episodes in this period, beginning with Mexico (1983 -1985) and spreading to Peru (1983), Uruguay (1983-1985), Chile (1985), Panama(1988-1989), El Salvador(1989-1991), Costa Rica(1987-1991). Simultaneously, several Sub-Saharan African nations, including Nigeria, Zambia, Ghana, Cote d'Ivoire, Sudan, and Tanzania, experienced severe sovereign debt and currency crises.
- Late 1980s: In the United States, the Savings and Loan Crisis (1988) resulted from financial deregulation and risky lending practices.
- Early 1990s - mid 1990s: The decade was marked by the Swedish banking crisis that lasted from 1992 to 1995, which emerged after the credit boom and housing market collapse. In Mexico, the Peso Crisis (1994-1996), also called the Tequila Crisis, triggered a severe recession across Latin America.
- Late 1990s: The Asian Financial Crisis began in Thailand (1997-2000) and spread rapidly to Indonesia (1997-2001), Malaysia (1997-1999), the Philippines(1997-2001), and South Korea (1997-1998).

Japan faced a parallel banking crisis (1997–2001) linked to long-standing bad loans from its earlier asset-price bubble. The Brazilian Currency and Banking Crisis lasted from 1996 to 1998.

- Early 2000s: Argentina’s Economic and Banking Crisis (2001–2003) followed the breakdown of its currency-board system.
- Late 2000s: The Global Financial Crisis, originating in the United States (2007–2011), became a worldwide banking collapse. Key affected economies included the United Kingdom (2007–2011), Switzerland (2008–2009), Iceland (2008–2011), Hungary (2008–2012), Ukraine (2008–2010), Mongolia (2008–2009), and Nigeria (2009–2012). The Icelandic Banking Crisis (2008–2012) also stands out as one of the most severe national collapses during this time.

Collectively, some of these episodes illustrate that banking crises are often regionally and temporally clustered, reflecting similar global shocks.

2.1.2 Data Preprocessing

The preprocessing procedure consisted of two key stages: data cleaning and feature engineering. Initially, only the variables relevant to the objective of the analysis were retained, while those deemed unnecessary were removed. The selected variables were:

- ISO country code (*isocode*)
- Year
- Binary target variable indicating whether a crisis occurred (*crisis0*)
- One-year lagged target variable (*crisis0_1*)
- Real GDP growth rate (*real_gdp_growth_ifs_1*)
- Logarithm of GDP per capita (*lgdp_per_capita_wdi_1*)
- Inflation (*deft_gdp_growth_wdi_1*)
- Real interest rate (*realirate_ifs_1*)
- Ratio of broad money (M2) to foreign exchange reserves (*m2_to_fereserves_wdi_1*)
- Growth rate of private credit (*domcrpriv_growth_wdi_1*)
- Ratio of non-financial assets to GDP (*ld_nfassets_to_gdp_wdi_1*)

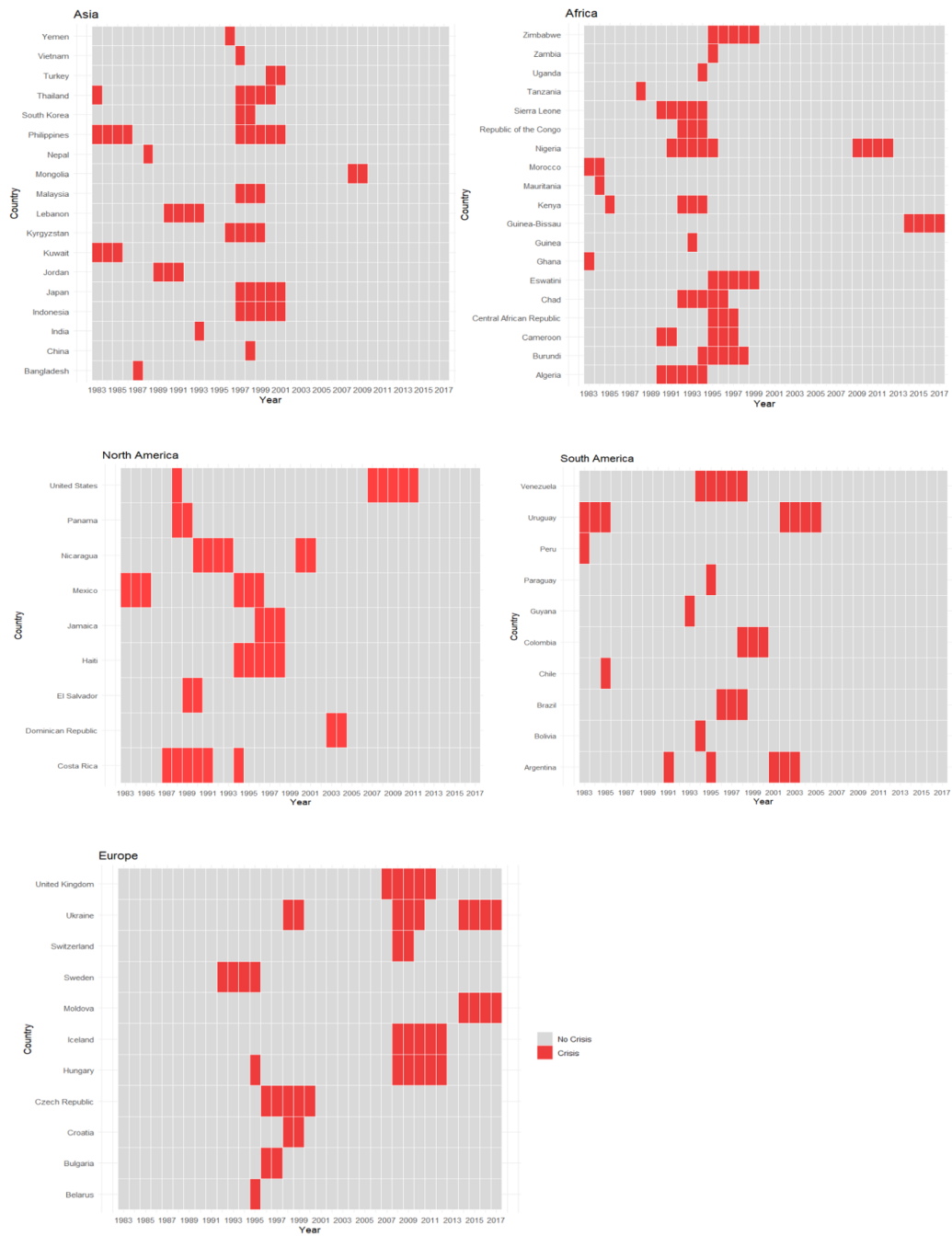


Figure 2.2: *Distribution of countries with respect to each continent that experienced at least one banking crisis between 1983 and 2017.*

This process helped simplify the dataset and reduce noise while preserving the variables most relevant for the analysis.

During the data cleaning phase, missing values were carefully handled to maintain dataset completeness and analytical consistency. The variable *Growth of net foreign assets to GDP* contained 160 missing values, for which the auxiliary variable (*aux1*) was created to flag these instances. Furthermore, if observations for a given country (across 129 countries from 1983 to 2017) were missing for a particular year, the corresponding row was added to ensure that each country had data for every year in the time range. In these added rows, the target variable *crisis* was set to 0, assuming that no crisis occurred in that period, while all covariates were assigned as missing (NA). To keep the information about the presence of missing values, another dummy variable *aux2* was created. This dummy variable was set to 1 when the corresponding covariates were missing and 0 otherwise. Consequently, the dataset is balanced, with each country represented by exactly one row per year, facilitating consistent longitudinal analysis across all observations. After these steps, the dataset contains 4,515 observations and 15 variables.

Following data cleaning, feature-engineering techniques were applied to enhance the temporal structure of the dataset. In particular, a rolling (lag) procedure with a fixed-five-year backward-looking window was implemented for selected variables, including the *crisis (lagged by one year)* indicator, *real GDP growth rate*, *logarithm of GDP per capita*, *inflation rate*, *real interest rate*, *M2-to-foreign-exchange reserves ratio*, *growth of private credit*, and *growth of net foreign assets to GDP*. For each country-year observation, the algorithm retrieves up to five available past records (including the current year), thereby generating five temporally ordered versions of selected variables and expanding the feature space. Particularly, in the initial years when fewer than five historical observations are present, the earliest available record for a country is repeated until the five-year window is completed. This mechanism ensures that all the observations are retained in a complete set of lagged features and that no rows contain missing values due to insufficient history.

2.1.3 Model Training

This model training phase prioritize the creation of a balanced, representative dataset to ensure fair cross-country analysis over time. A primary challenge in crisis modeling is the inherent sparsity of the target variable; many countries never experienced a crisis, which can lead to a loss of representativeness during the down-sampling process. To mitigate this, the data preparation strategy centered on grouping similar countries based on geographic and economic characteristics. The intuition behind this approach was to ensure that at least one representative country from

every group remained present in both the training and test subsets after down-sampling.

To facilitate this grouping, metadata was obtained via web scraping for each ISO code of the country and integrated into the main dataset. This enriched the data with contextual variables, including continent and land area. The categorization process followed several key steps:

- **Geographic Aggregation:** Each country was assigned to one of the five continent groups: *South America*, *Europe*, *Asia*, *Africa* and *North America and Oceania*. The latter two continent were merged specifically because Oceania contained no recorded crisis cases; keeping it isolated would have made it impossible to include it in a balanced split.
- **Variable Binning:** To further reduce heterogeneity within these broad regions, a binning procedure was applied to two continuous variables: land area ($LandArea(km^2)$) and GDP per capita ($Growth\ of\ net\ foreign\ assets\ to\ GDP$). By dividing these into two bins within each continent, the model ensures that comparison are made between countries of similar regional and economic scales.
- **The Stratification identifier:** Using the continent, country name and the bin identifiers for both land area and GDP, a composite grouping variable named *sel* was constructed (*e.g.* observations are recorded as: *Asia.Japan.1.2*). This variable uniquely identified each subset of observations, allowing for a stratified under-sampling approach.

Once the groups were defined, the dataset was split into training and testing subsets through a balanced under-sampling procedure. For each group defined by the *sel* variable, the number of observations belonging to the crisis class ($Crisis = 1$) and the non-crisis class ($Crisis = 0$) was calculated. If one of the classes were absent (*i.e.*, rows belonging to that group, either crisis or non-crisis), they were set aside for the external dataset labelled as "test3" (3,188 observations) to avoid introducing single-class groups into the training set or the modeling process. When a group contained only one observation in each class, both records were assigned directly to the training set ("balanced_train") since a meaningful split into training and testing portions was not possible.

For all remaining groups containing sufficient observations in both classes, a random sampling procedure was applied to balance classes by matching the frequency of the minority class within each group. From this balanced sample, approximately 30% of observations were randomly selected for testing subset ("balanced_test", 116 observations), while 70% were assigned to the training subset ("balanced_train", 342 observations). Any

observations not selected during this sampling step were placed in an auxiliary subset called "test2" (860 observations). This procedure ensured that both the training and testing subsets contained an equal number of crisis and non-crisis observations within each group while preserving unused data for potential external validation.

The result of this process was a dataset with perfectly balanced target classes in both the training and testing groups. Specifically, the training set contained 171 crisis and 171 non-crisis observations, while the testing set contained 58 crisis and 58 non-crisis cases. This stratified and balanced data-splitting strategy was chosen over a simple random split to address the key challenges: class imbalance and heterogeneity across countries and regions. In crisis prediction, typically, crisis events are outnumbered by the non-crisis case, which can lead models to favour the majority class and yield misleadingly high accuracy but poor sensitivity to crises. By undersampling the majority class and ensuring the equal representation of both classes within each continental and economic grouping, the model was trained to detect crisis patterns more effectively. Moreover, grouping by continent and binning by land area, and GDP per capita reduced the influence of structural differences between countries, allowing the model to learn more general relationships rather than being biased toward specific regions or economic sizes.

2.2 Research Design

The overall research design of the code is structured around a systematic evaluation of multiple machine learning pipelines for classification tasks using supervised learning. The approach is rooted in the principles of AutoML, particularly as implemented by the TPOT AutoML algorithm, which automatically generates and optimizes pipelines combining preprocessing, feature selection, and classifiers. This work further compares different machine learning pipelines systematically found via the TPOT AutoML algorithm and documents their performance. It includes tree-based models (*e.g.*, ExtraTreesClassifier, GradientBoostingClassifier) [26], boosting methods (XGBClassifier) [14], linear models (LinearSVC, SGDClassifier) [46], probabilistic models (BernoulliNB) [25], and stacking ensemble models are used to integrate diverse classifiers into a unified pipeline so that their combined predictions produce a stronger model than any individual method alone. Each pipeline also integrates feature engineering and preprocessing steps, such as normalization, binarization, recursive feature elimination, or percentile-based selection, to improve predictive performance. The evaluation approach is iterative and reproducible. Finally, the results are documented both quantitatively and visually.

2.2.1 Using TPOT for AutoML

The Tree-based Pipeline Optimization Tool (TPOT) is one of the most widely used AutoML frameworks. Built on top of scikit-learn [59], TPOT automates the design of end-to-end pipelines that may include data preprocessing, feature engineering, model selection, and ensemble construction [50]. Figure 2.3 depicts a possible pipeline identified by TPOT. Each element in the pipeline is a computational element performing one of the mentioned operations. Initially, each pipeline starts with one or more input nodes, which refer to the input dataset. These copies are then supplied to the pipeline operators of the preceding figure. As shown in the figure, the identified elements are not limited to PCA and Polynomial feature preprocessing operators, but also the following elements can be used: RandomizedPCA, PolynomialFeatures, Binarizer, StandardScaler, MinMaxScaler, MaxAbsScaler, and RobustScaler.

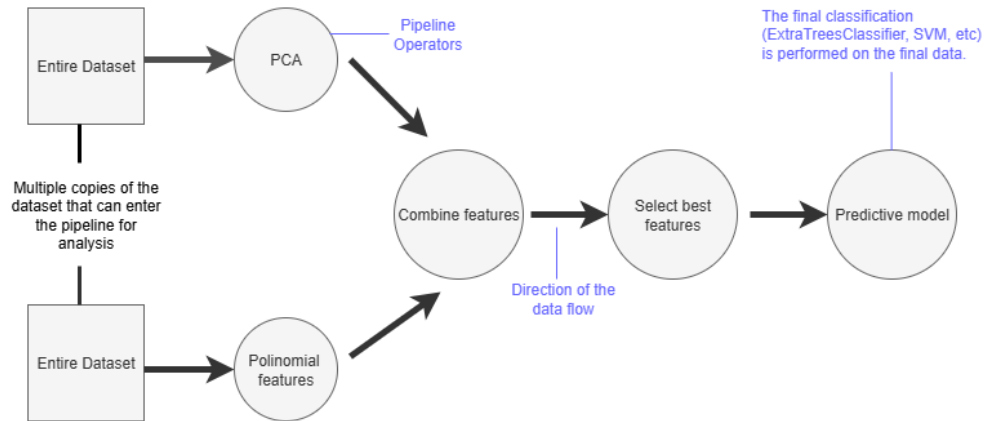


Figure 2.3: *The general workflow of TPOT pipeline. As the data set passes through the pipeline operators, the features are added, deleted, or altered. Finally, the combined dataset is fed to the final classifier to perform the final classification [50].*

The next step in building a predictive model is to select features that contribute most to the target variable while removing irrelevant or redundant features. Reducing the dimensionality of the dataset not only improves computational efficiency but also enhances the ability to generalize to new data. TPOT is designed to construct, evaluate, and optimize machine learning pipelines automatically, combining preprocessing, feature selection, model fitting, and hyperparameter tuning. TPOT operates on the training dataset provided by the user and evaluates candidate pipelines using cross-validation, ensuring robust estimation of performance and minimizing overfitting [50].

TPOT employs genetic programming (GP) to evolve pipelines over successive generations. This is a well-known evolutionary computation technique that allows TPOT to efficiently navigate vast search spaces and

discover combinations of preprocessing and modelling steps [54]. The three main operations of a genetic algorithm are: selection, mutation, and crossover. The preceding Figure 2.4 shows the workflow of the proposed method. The process begins with the initialization of a population of candidate pipelines and then proceeds with the evaluation and selection phase. Each pipeline is trained and validated using five-fold cross-validation, with performance measured by a user-defined metric; in this study, the F1 score was adopted. Based on these results, the highest-performing pipelines are selected as parents for the next generation. The subsequent phases of crossover and mutation are responsible for creating new pipelines and maintaining diversity within the population. In a crossover, segments of two parent pipelines are combined to generate a new pipeline that inherits properties from both. Mutation introduces further variability by randomly modifying parts of a pipeline, either by adjusting hyperparameters (such as the depth of a decision tree or the number of estimators in a random forest) or by replacing an operator with another compatible alternative.

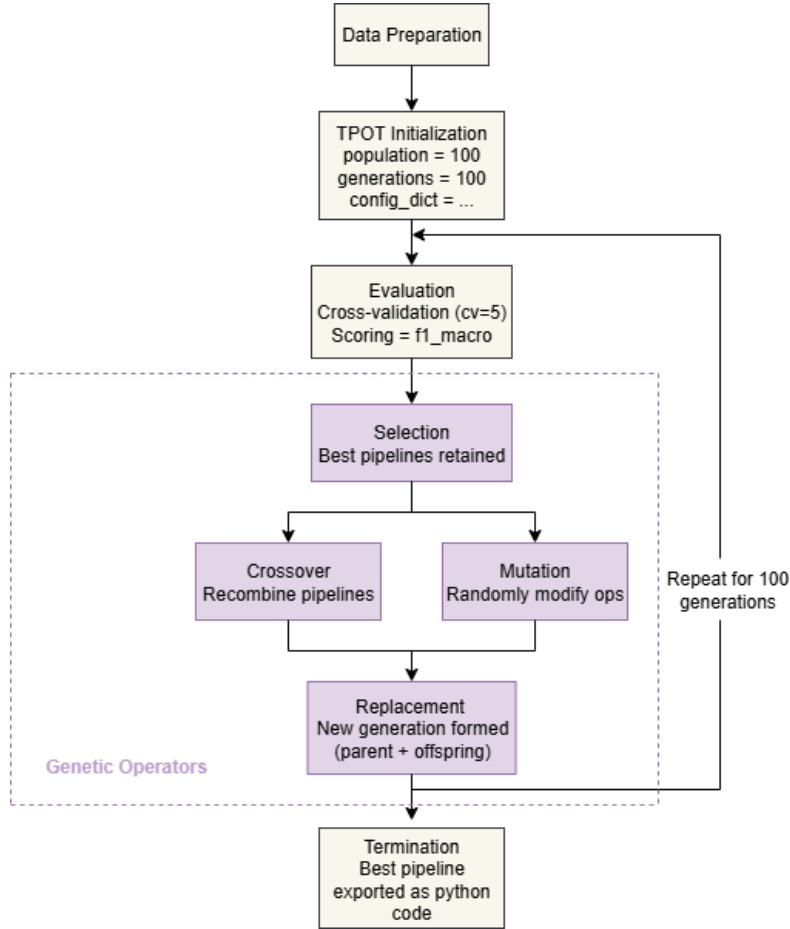


Figure 2.4: *Workflow of TPOT Pipeline Optimization in the proposed method.*

Following these operations, the resulting population N includes both the

parents and newly generated offspring to keep the population within the N size, and the less performing pipelines are dropped. This optimization process was conducted for N generations, resulting in the evaluation of approximately 10,000 pipelines. The optimization terminates once the predefined number of generations has been completed. At this stage, TPOT identifies and outputs the single best-performing pipeline discovered throughout the search. The optimal pipeline is exported as executable Python code, which allows for reproducibility and further analysis.

2.2.2 Machine Learning Models

In recent years, machine learning techniques have gained traction in financial crisis prediction. The application of machine learning models has been further promoted to play a central role in modern data-driven research, providing powerful tools for pattern recognition, prediction, and decision-making across diverse domains. The fundamental objective of the machine learning paradigm is to equip computers with the ability to learn interdependently without human intervention [60].

The foundational taxonomy within machine learning categorizes algorithms into three groups based on the type of supervision signal that is accessible during the training phase: supervised, unsupervised, and semi-supervised learning. Supervised learning operates on a corpus of labelled training data, where the algorithm learns a mapping function from input features to known output labels, with the objective of generalizing this function to accurately predict labels for novel, unseen instances. Throughout the training phase, humans must provide the proper input and output, as well as feedback on the prediction accuracy for supervised algorithms. In contrast, unsupervised learning infers patterns from entirely unlabelled data, seeking to uncover intrinsic structures, such as inherent groupings (clustering) or simplified representations (dimensionality reduction), without explicit guidance. Bridging these two approaches, semi-supervised learning leverages a small set of labeled data in conjunction with a larger pool of unlabelled data.

This study shows that constructing a manual pipeline with different types of supervised machine learning algorithms inspired by TPOT architecture, rather than relying solely on the automated output, offers a distinct advantage in operational efficiency and decision-making reliability. While TPOT is an excellent tool for discovering high-performing mathematical configurations, its genetic search process can produce overly complex architectures that are difficult to interpret in a policy context. By manually rebuilding the pipeline using the same training and test datasets, you refine the model into a deterministic and stable framework. This approach improves decision-making efficiency by:

- **Enhancing Interpretability:** It allows selecting and stacking specific supervised machine learning algorithms that provide a transparent risk signal, moving away from a "black-box" result to a system where the logic behind a crisis alert is clearer than an automated process.
- **Ensuring Structural Consistency:** A manual pipeline eliminates the stochastic (randomized) variance inherent in AutoML searches, providing a fixed benchmark that yields consistent results across different runs, a mandatory requirement for high-stakes economic interventions.
- **Filtering Economic Noise:** By curating the specific learners and stacking methods, the manual pipeline more effectively isolates actionable early-warning signals from high-dimensional noise, ensuring that decision-makers are not distracted by statistical artifacts that do not reflect real-world economic risk.

Supervised machine learning is broadly divided into classification and regression tasks, with classification focusing on predicting categorical outcomes using labeled data [35]. The following classification models are included in this study:

- *Linear models* (e.g., LinearSVC, SGDClassifier) remain widely used for their simplicity and interpretability when classes are approximately linearly separable.
- *Probabilistic models* (e.g., Naive Bayes, Bayesian Networks) use the principles of probability theory to make predictions or decisions. Instead of just outputting a class label, they estimate the likelihood of each possible outcome given the observed data.
- *Tree-based models* (e.g., Decision Trees, Random Forests) capture non-linear relationships through recursive partitioning of the feature space.
- *Boosting models* (e.g., XGBoost, AdaBoost) improve predictive performance by iteratively correcting errors from weak learners.
- *Ensemble models* combine multiple learners to enhance accuracy and robustness, demonstrating that leveraging diversity among classifiers improves generalization.

However, designing optimal pipelines for supervised machine learning algorithms often requires careful experimentation with preprocessing, feature selection, and hyperparameter tuning, which can be time-consuming and prone to human bias. Automated machine learning (AutoML) frameworks such as TPOT (Tree-based Pipeline Optimization Tool) address

this challenge by using genetic programming to automatically generate, evaluate, and refine machine learning pipelines. TPOT explores a rich search space of transformations, models, and ensembles, evolving candidate pipelines over successive generations to maximize performance metrics like accuracy or F1 score. This makes TPOT particularly effective for supervised learning tasks such as classification, where finding the best combination of models and preprocessing steps is critical for robust predictive performance.

2.2.2.1 Multiple linear regression model

Linear models assume that the association between input features and the target variable can be expressed through a linear function [28]. Within this family, Support Vector Machines (SVMs) and Stochastic Gradient Descent (SGD) classifiers are among the most commonly used approaches.

Support Vector Machines (SVMs)

SVM is one of the most used ML algorithms for classification and regression tasks due to its theoretical grounding and strong generalization ability. Given a training dataset $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_l, y_l)\}$, where $x_i \in \mathbb{R}^n$ is an instance with n features and y_i are either 1 or -1 , indicating class labels ($y \in \{1, -1\}$). The fundamental concept of SVM lies in finding an optimal hyperplane that maximizes the margin between different classes in a dataset, Figure 2.5.

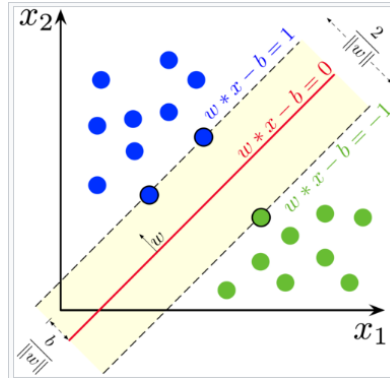


Figure 2.5: An example of the separable problem in a two dimensional space. The support vectors are outlined with black circle on the margin. The optimal hyperplane maximizes the separation between two classes [43].

Depending on the nature of the training dataset, two main cases are considered: hard margin and soft margin [15]. In the hard margin formulation, it is assumed that the dataset is linearly separable, meaning there exists at least one hyperplane that can separate the two classes

without any classification errors. The SVM aims to find the optimal hyperplane that maximizes the margin between the support vectors of each class. Let $x_i \in \mathbb{R}^p$ (reference to Equation 2.1) denote the feature vector of the i -th observation and $y_i \in -1, +1$ its corresponding class label. To allow for potential non-linear separation, the original features may be mapped into a higher-dimensional space through a transformation function ϕ . In this feature space, the separating hyperplane is defined by a weight vector w and a bias term b . Formally, the hard-margin constraint is expressed as:

$$y_i (w^T \phi(x_i) + b) - 1 \geq 0 \quad \forall i \quad (2.1)$$

Moreover, in many real-world problems, the classes are not only noisy but also inherently nonlinearly separable. In such cases, enforcing a hard margin can lead to poor performance, since even a small amount of noise may prevent the existence of a valid separating hyperplane. To handle non-linear decision boundaries, SVM uses the kernel trick, which implicitly maps the input data into a higher-dimensional feature space without explicitly computing the transformation. A kernel function denotes an inner product in a feature space; it measures the similarity between any pair of inputs x_i and x_j , and is usually denoted as $K(x_i, x_j) = [\phi(x_i), \phi(x_j)]$. Common kernels include the linear kernel, polynomial kernel, and the Radial Basis Function (RBF) kernel.

[15] introduced the soft margin formulation to address these challenges. The soft margin allows some flexibility by introducing slack variables $\xi_i \geq 0$, which permit certain data points to lie inside the margin or even on the wrong side of the hyperplane. The modified constraint in (2.2) is:

$$y_i (w^T \phi(x_i) + b) - 1 + \xi_i \geq 0 \quad i = 1, \dots, l, \quad (2.2)$$

$$\xi_i \geq 0 \quad i = 1, \dots, l$$

The optimization problem now becomes a trade-off between maximizing the margin and minimizing the classification errors, controlled by the regularization parameter C in (2.3):

$$\min \frac{1}{2} \|w\|^2 + C \sum_{i=1}^l \xi_i. \quad (2.3)$$

Here, the parameter C is a regularization term that induces a penalty for misclassifications.

Despite these advantages, SVM has limitations. The choice of kernel and

hyperparameters significantly influences model performance and requires careful tuning, often via cross-validation.

Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent (SGD) is a widely used optimization algorithm in machine learning, particularly effective for large-scale and high-dimensional data, such as financial and macroeconomic indicators used in crisis prediction. Unlike traditional gradient descent, which computes the gradient of the cost function using the entire dataset, SGD updates model parameters using only a single observation (or a small batch of observations) at each iteration. This leads to faster convergence and scalability, which is crucial when analyzing financial time series or global economic data spanning multiple countries and decades. Figure 2.6 shows how the parameters change while performing gradient descent in a two-dimensional space.

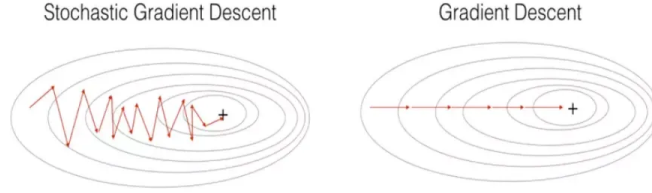


Figure 2.6: *Comparison of optimization trajectories for Stochastic Gradient Descent (left) and Batch Gradient Descent (right). The contour lines represent level sets of the objective (loss) function. The red paths illustrate the sequence of parameter updates toward the optimum. Stochastic Gradient Descent follows a noisy, zig-zag path due to gradient estimation from mini-batches, whereas Batch Gradient Descent moves smoothly along the steepest descent direction toward the minimum [53]*

The general objective of supervised learning for financial crisis prediction is to minimize a loss function $L(\theta)$ (reference to Equation (2.4)), where θ represents the model parameters. For instance, if we model the probability of a crisis occurrence using logistic regression, the loss function can be written as the negative log-likelihood:

$$L(\theta) = - \sum_{i=1}^N [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)], \quad (2.4)$$

where $y_i \in \{0, 1\}$ denotes class labels whether a financial crisis occurred in observation i , and $\hat{p}_i = \sigma(x_i, \theta)$ is the predicted probability with σ being the logistic function.

The update rule for the traditional gradient descent algorithm, considering a single parameter θ is (2.5):

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} L(\theta^{(t)}), \quad (2.5)$$

where $\theta \in \mathbb{R}^p$ in (2.6) denotes the vector of model parameters (e.g., regression coefficients and intercept) to be estimated, t represents the iteration step, $L(\theta)$ is the loss function $\nabla_{\theta} L(\theta)^{(t)}$ is the gradient of the loss function with respect to the parameter vector evaluated at iteration t , and $\eta > 0$ is the learning rate controlling the step size of the update. However, for large financial datasets, computing the full gradient can be computationally expensive. SGD approximates this by updating the parameters using only one randomly selected observation (or mini-batch) at each iteration:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} \ell(\theta^{(t)}; x_i, y_i), \quad (2.6)$$

where $\ell(\theta; x_i, y_i)$ is the loss function evaluated on the i -th observation.

One of the challenges of SGD is noisy convergence. Since the gradient is estimated based on a single small batch, the updates can be noisy, causing the cost function to fluctuate and slower convergence rather than steadily decrease. In practice, variants such as SGD with momentum, RMSProp, or Adam [67] are often employed to stabilize convergence and accelerate learning when working with complex, nonlinear predictors of crises.

In the context of financial crisis prediction, SGD allows models to handle unbalanced datasets, high-dimensional macroeconomic indicators, and large historical records efficiently ([64], [24]).

2.2.2.2 Probabilistic Models

Probabilistic models, such as Naive Bayes, classify data based on probability distributions derived from the features. These models are computationally efficient and scale well to large datasets.

Bernoulli Naive Bayes

The Naive Bayes(NB) algorithm assumes the conditional independence between features given the class label [30]. In other words, a naive Bayes model implies that there is no information shared among the predictors and that the class information provided by one variable is independent of others. The classifier is named after this assumption due to its unrealistic nature. NB is a conditional probability model, at its core, the algorithm leverages Bayes's rule, denoted in (2.7):

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)}, \quad (2.7)$$

where C denotes the class label (*e.g.*: crisis or no crisis), $X = (X_1, x_2, \dots, x_n)$ is the feature vector of financial indicators, $P(C|X)$ is the posterior probability of a crisis given the observed features, $P(X|C)$ is the likelihood of the feature under a class C and $P(C)$ is the prior probability of the class. Bernoulli NB is a subcategory of the NB Algorithm. It is particularly used when the predictors are binary. In this case, the likelihood function in (2.8) is modeled as a product of Bernoulli distributions [68].

$$P(X | C) = \prod_{i=1}^n \left[P(x_i = 1 | C)^{x_i} \cdot (1 - P(x_i = 1 | C))^{1-x_i} \right] \quad (2.8)$$

where $x \in \{0, 1\}$ indicates the presence or absence of a feature and $P(x_i = 1|C)$ is the probability if the feature i is present given class C . This probabilistic formulation allows the model to combine multiple binary signals into an overall prediction. The prediction rule in (2.9) is then defined as:

$$\hat{C} = \underset{C \in \{crisis, no-crisis\}}{\operatorname{argmax}} P(C) \prod_{i=1}^n P(x_i, C). \quad (2.9)$$

The naive Bayes classifier combines the model with this decision rule to classify an observation into the class with the highest posterior probability, in order to minimize the probability of misclassifications.

Despite its simplicity, BernoulliNB has been found effective in domains with high-dimensional and sparse data, such as text classification, and its application to early warning systems in economics is motivated by its interpretability, computational efficiency, and ability to handle binary transformations of continuous indicators [2].

2.2.2.3 Tree-based Models

Tree-based models, such as Decision Trees, Random Forests, and Gradient Boosted Trees [26], classify data by splitting it into subsets based on feature values. Each internal node of the tree represents a decision on a feature, and the leaves represent class predictions. These models are interpretable, can handle both categorical and continuous variables, and naturally capture non-linear relationships.

Gradient Boosting

Gradient Boosting (GBM) [47] is a powerful ensemble method that builds predictive models in a stage-wise manner by iteratively fitting decision trees using the negative gradients of a specified loss function [5]. It trains

models sequentially, with each new model attempting to fix the mistakes of its predecessor.

In the formulation of GBM, the fundamental goal is to seek the function $F(x)$ that better approximates best the output variables from the input variables, therefore we seek a function $F(x)$ that minimizes the expected value of a differentiable loss function $L(y, F(x))$ in (2.10):

$$F^*(x) = \arg \min_F \mathbb{E}_{y|x} [L(y, F(x))] \quad (2.10)$$

At each boosting iteration m , the model is updated as shown in (2.11).

$$F_m(x) = F_{m-1}(x) + v \cdot h_m(x), \quad (2.11)$$

where $v \in (0, 1]$ is the learning rate and $h_m(x)$ is a base learner (typically a regression tree) fitted to the negative gradient of the loss function in (2.12) with respect to the current prediction (*pseudo-residuals*):

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)} \quad (2.12)$$

For probabilistic modeling, the choice of loss function shifts from mean-squared error to a proper scoring rule that captures distributional properties. For example (see (2.13)), when predicting the probability of a financial crisis (binary outcome), the logistic loss is commonly used:

$$L(y, p(x)) = -[y \log p(x) + (1 - y) \log(1 - p(x))] \quad (2.13)$$

where $p(x) = \sigma(F(x))$ is the predicted probability of crisis, with $\sigma(z) = \frac{1}{1+e^{-z}}$ being the logistic function. This allows the boosting algorithm to output probability estimates rather than deterministic class labels, which can be interpreted as the likelihood of a financial crisis occurring given macroeconomic and financial indicators. By adopting probabilistic Gradient Boosting, researchers and practitioners can move beyond point forecasts and incorporate measures of predictive uncertainty.

Extra Trees (Extremely Randomized Trees)

Probabilistic Extra-Trees (Extremely Randomized Trees) is an ensemble learning technique introduced [27], designed for both classification and regression tasks. It is very similar to the Random Forest Classifier and only differs in the construction of the decision trees in the forest. Random Forest optimizes the split points computing via locally optimal cut-point, like Gini impurity or mean squared error (MSE), while Extra-Trees incorporates additional randomness. Extra trees have two main differences

in relation to Random Forest, too: (1) each tree is trained using the whole learning sample (rather than a bootstrap sample), and (2) the top-down splitting is randomized: for each feature under consideration, only a number of random cut-points are selected.

The Extra-Trees splitting procedure for numerical features is as follows: formally, an Extra-Trees ensemble consists of M trees, each constructed on the full dataset. At each node, the number of features $\{f_1, \dots, f_k\}$ is randomly selected. Then, for each feature f_i , a number of thresholds $\{\lambda_{j,1}, \dots, \lambda_{j,r}\}$ are sampled uniformly at random from the empirical range of that feature. Among all randomly generated splits, the one yielding the best score (e.g., highest reduction in impurity or error) is chosen to split the node [58]. This process is used several times with the (full) original learning sample to generate an ensemble model (we denote by M the number of trees of this ensemble). The final prediction is obtained using the arithmetic average in regression problems, as shown in (2.14), such as predicting quantitative crisis indicators:

$$\hat{y}(x) = \frac{1}{M} \sum_{i=1}^M t_i(x) \quad (2.14)$$

where $t_i(x)$ is the prediction of tree i on input x .

According to the bias-variance perspective, increasing randomization (e.g., more random splits per node) tends to decrease variance but may increase bias, which must be carefully balanced depending on dataset characteristics [27]. In high-dimensional or noisy datasets, a common scenario in financial crises, modeling Extra-Trees' aggressive randomness can help against overfitting to spurious patterns, producing more robust predictions. However, overly random splits may still degrade predictive accuracy, so hyperparameters like the number of random thresholds per feature and the number of features considered per node must be tuned.

In the context of financial crisis prediction, where data is often noisy and non-stationary, this bias variance trade-off can be particularly beneficial. The ability to generalize well to unseen patterns while mitigating overfitting makes probabilistic Extra-Trees appealing for stress-sensitive forecasting models.

2.2.2.4 Boosting models

XGBoost

XGBoost, or eXtreme Gradient Boosting [66], is a high-performance implementation of gradient boosting that constructs an ensemble of decision trees in a sequential manner. To enhance the model's performance, XGBoost sequentially combines decision trees, which serve as its base

learners. Each new tree is added to correct errors made by the ensemble so far, making it an effective predictor for complex, structured data. Besides its ensemble approach, its integrated parallel processing enables speedy model training on big datasets through optimizations such as second-order approximation, regularization, and efficient tree construction strategies. The generic iterative process of the XGBoost algorithm often starts by setting the prediction to zero. Each subsequent tree is then added to minimize mistakes. For a prediction of $\hat{y}_i$ for an instance i using an additive ensemble of K regression trees is shown in (2.15):

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i), \quad f_k \in S \quad (2.15)$$

where each f_k represents an independent regression tree belonging to the functional space S . This additive process is iterative. Starting from $\hat{y}_i^{(0)} = 0$ at each subsequent iteration t , a new tree f_t is trained and added to the ensemble to minimize the objective function. The prediction at iteration t is shown in (2.16):

$$\hat{y}_i^t = \hat{y}_i^{t-1} + f_t(x_i) \quad (2.16)$$

This continues until the ensemble sufficiently minimizes the error.

The learning objective in (2.17) combines a task-specific loss $l(y_i, \hat{y}_i)$, which measures the discrepancy between the true target value y_i and the predicted value $\hat{y}_i$, with a regularization term that penalizes model complexity. Formally, the objective function is defined as:

$$obj(\theta) = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k) \quad (2.17)$$

where n denotes the number of observations and K the number of decision trees and $\Omega(f_k)$ typically includes a penalty for the number of leaves T and the leaf weights w_j shown in (2.18):

$$\Omega(f_t) = \gamma T_k + \frac{1}{2} \lambda \sum_{j=1}^T \|w_k\|^2. \quad (2.18)$$

Where T_k is the number of leaves in tree k , w_k represents the vector of leaf weights, $\gamma \geq 0$ penalizes the number of leaves (discouraging overly deep or complex trees), and $\lambda \geq 0$ is an L_2 regularization parameter that penalizes large leaf weights. This regularization encourages simpler trees and helps prevent overfitting.

2.2.2.5 Ensemble Methods

In this study, ensemble learning techniques are employed to enhance predictive performance by combining multiple classifiers within structured pipelines. The general framework of any ensemble method is to leverage a set of baseline classifiers $c_1, c_2, \dots, c_l$ to make an aggregate prediction. The effectiveness of an ensemble framework is primarily dictated by three influential characteristics, which are discussed in further detail in this section:

- The Nature of the Baseline Classifiers
- Dependency Between Baseline Models
- Fusion Methods

The Nature of the Baseline Classifiers

The ensemble strategies implemented in the pipelines can be broadly categorized into homogeneous and heterogeneous ensembles, based on the different baseline classifiers. A homogeneous ensemble is composed of multiple base learners that belong to the same model family, while a heterogeneous ensemble integrates different types of learners within a single framework. In this work, the homogeneous approach is first explored. E.g., the first pipeline employs the Extremely Randomized Trees (Extra-Trees) and Decision Tree classifiers as its sole ensemble methods. In these cases, all base learners belong to the same family of tree-based methods, and the ensemble strength derives from repeated sampling, randomized splits, or boosting iterations. Likewise, several other pipelines have incorporated the heterogeneity through different learning algorithms.

Dependency Between Baseline Models

The diversity of the baseline classifiers is the second influential factor in the ensemble system. This factor concerns how the classifiers are trained either sequentially or in parallel [32]. In sequential ensembles, the base learners are trained one after another, where each subsequent model is trained to correct the errors made by its predecessor. This sequential dependency reduces bias and improves accuracy, but it also makes the process more computationally intensive. In contrast, parallel ensembles, such as bagging and random forests, construct multiple base learners independently and then aggregate their predictions. This parallel design reduces variance and enhances stability, while allowing for faster training since models can be built simultaneously. Figure 2.7 depicts frameworks of the two techniques.

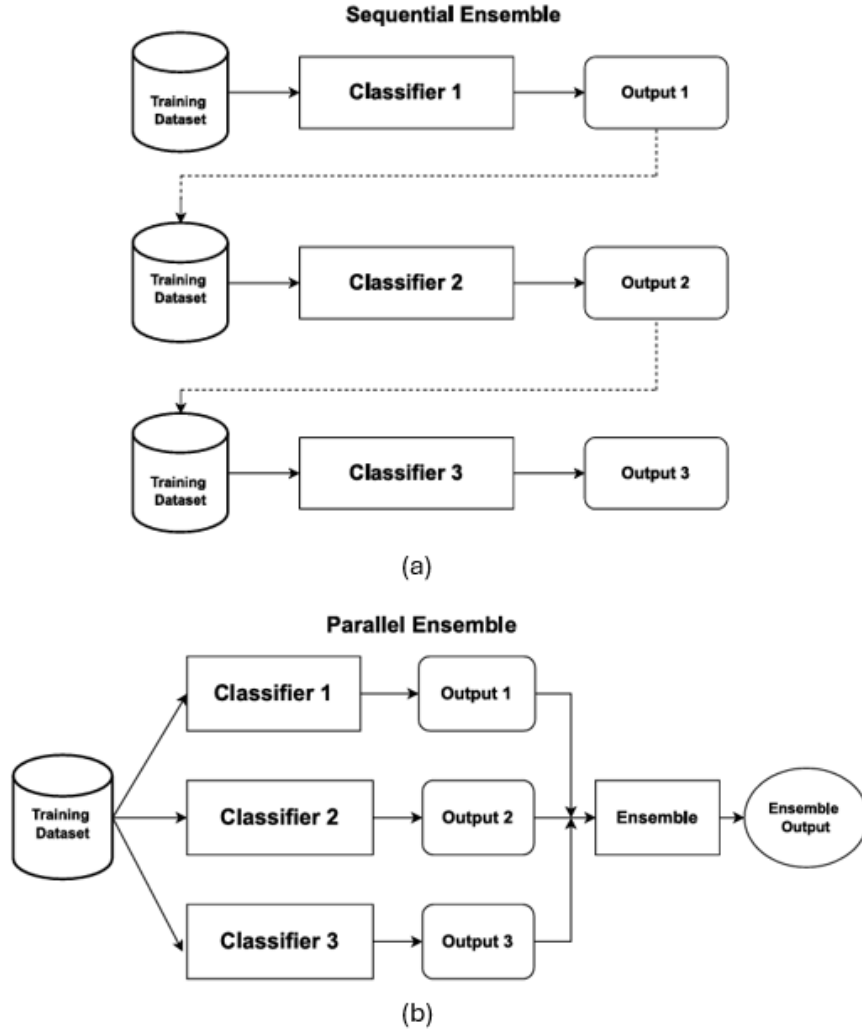


Figure 2.7: *General Framework of (a) sequential ensemble and (b) parallel ensemble*

Fusion Methods

There are three main types of ensemble methods: Bagging/Bootstrap aggregation, Boosting, and stacking/ stacked generalization.

Bagging

The bagging method [6], also known as bootstrap aggregating, is designed to reduce model variance by training multiple base learners on different bootstrap samples of the data. Given a dataset $D = \{(x_i, y_i)\}_{i=1}^n$, bagging generates B bootstrap subsets $D^{(1)}, D^{(2)}, \dots, D^{(B)}$ by sampling with replacement. Each subset is used to train a base learner $h_b(x)$. For classification, the final prediction is obtained using the majority voting fusion method denoted in (2.19).

$$H(x) = \text{model}\{h_1(x), h_2(x), \dots, h_B(x)\}, \quad (2.19)$$

and the function for bagging denoted as (2.20):

$$H(x) = \frac{1}{B} \sum_{b=1}^B h_b(x), \quad (2.20)$$

where $h_b(x)$ weak learners, $\frac{1}{B}$ generates bootstrapping sets.

An example of bagging is the Random Forests (RF) method [7]. There are several challenges to implementing the bagging method: determining the optimal number of base learners and subsets, the maximum number of bootstrap samples per subset, and the type of fusion method for the integration of the baseline classifiers from various voting methods. The main advantages of the bagging method are the reduction of variance, the elimination of overfitting, and the ability to perform well on high-dimensional data. The drawbacks of bagging are the high computational effort required, high bias, and the lack of interpretability [10].

Boosting

Boosting is a sequential ensemble technique that reduces bias by iteratively training weak learners, each focusing on correcting the errors made by the previous learners [20]. At each iteration t , a weak learner $h_t(x)$ is trained on a weighted dataset. Initially, all instances have equal weight $w_i^{(1)} = \frac{1}{n}$. After training, the weights are updated so that misclassified instances receive higher weights. The function of the final boosted model is a weighted sum of all learners:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right), \quad (2.21)$$

Here $H(x)$ creates a strong classifier from several weak classifiers $h_t(x)$ and the α_t is the weight assigned to learner h_t , often based on its classification accuracy *i.e.*, creating a second model that attempts to correct the errors from the first model. Boosting, like bagging, can be used for regression and classification problems. Adaptive Boosting (AdaBoost) [21], Stochastic Gradient Boosting (SGB) [23], and Extreme Gradient Boosting (XGB), also known as XGBoost [22], are examples of boosting algorithms. Despite its predictive power, boosting presents several key challenges:

- **Scalability of Sequential Training:** Unlike parallel methods, where models are built simultaneously, boosting is inherently sequential.

Because each model h_t depends on the results of h_{t-1} , the process cannot be easily distributed across multiple processors. This creates a computational bottleneck, as the training time scales linearly with the number of iterations, making it difficult to apply to massive datasets without significant optimization.

- **Sensitivity to Overfitting:** As the number of iterations increases, the ensemble may become overly specialized to the training data. If the base learners are too complex or the iterations are too numerous, the model begins to memorize noise and outliers rather than general patterns.
- **Computational Expense and Complexity:** Boosting algorithms often require the fine-tuning of a large number of hyperparameters (such as learning rate, tree depth, and regularization), which requires more intensive memory and processing power.

Boosting offers the advantage of the interpretability of the model and the reduction of the variance and the bias in a machine learning ensemble. One of the main drawbacks of boosting is that each classifier must fix the errors in the predecessors.

Stacking

Stacking method [61], also known as Stacked Generalization, is an ensemble model that uses parallel ensemble techniques where baseline learners are trained simultaneously. It combines the predictions of multiple heterogeneous base learners through a meta-learner(new model). Let $h_1(x), h_2(x), \dots, h_M(x)$ be predictions from M base learners. Unlike bagging and boosting, stacking does not rely solely on averaging or weighting; instead, the predictions of base models are used as input features for a higher-level model, and stacking forms a new dataset $Z = \{(z_i, y_i)\}$, where

$$z_i = h_1(x_1), h_2(x_2), \dots, h_M(x_i),$$

A meta-learner $g(z)$ is then trained on this new dataset to produce the final prediction :

$$H(x) = g(h_1(x_1), h_2(x_2), \dots, h_M(x_i)).$$

The primary challenge in Stacking (Stacked Generalization) lies in the identification of appropriate baseline models that can generate better predictions from the datasets. As the volume of available data increases, the architectural complexity of these multi-layered systems grows significantly. This high computational burden often results in a "black-box" effect: when a meta-learner is tasked with synthesizing outputs

from dozens of high-dimensional baseline models, it becomes increasingly difficult to trace how specific input features influence the final prediction. Consequently, this high computational effort directly hinders model interpretability, making it hard for practitioners to explain the logic behind the ensemble’s decisions. Furthermore, while stacking is a powerful general framework, it is susceptible to the curse of dimensionality and overfitting, particularly when the meta-learner is trained on a high-dimensional feature space created by numerous base models. Despite these challenges, stacking methods provide a more comprehensive representation of the underlying data patterns by capturing the unique strengths of various algorithms, ultimately making the final predictive model both more precise and more effective.

2.2.3 Evaluation Metrics

Since the dataset contains an equal number of crisis and non-crisis observations, accuracy can be used as a reliable primary performance indicator. However, relying solely on accuracy may overlook nuances in classification errors. Therefore, additional metrics: precision, recall, and F1-score are also used to ensure a comprehensive understanding of the performance of the model.

Accuracy

Accuracy represents the proportion of correctly predicted instances relative to the total number of observations. It provides a general indication of the overall performance of the model by quantifying how often the classifier produces the correct prediction. Although widely used due to its simplicity, accuracy may not fully reflect the quality of the model when the cost of misclassification differs or when class distributions are highly uneven.

Mathematically, accuracy is expressed as:

$$Accuracy = \frac{Total\ Correct\ Predictions}{Total\ Number\ of\ Observatons} \quad (2.22)$$

Precision

Precision (also known as Positive Predictive Value) measures the proportion of predicted positive cases that are truly positive. It focuses on the correctness of positive predictions and indicates how reliable the classifier is when identifying a specific class. High precision implies that the model generates very few false positives, making it particularly relevant in contexts where false alarms are undesirable (*e.g.*, predicting a financial crisis when none exists).

In a similar way, the same logic can be applied to the negative class to de-

termine the Negative Predictive Value (NPV). While standard precision focuses on the reliability of positive labels, NPV measures the proportion of predicted negative cases that are truly negative. This metric indicates the reliability of the classifier when it predicts that an event will not occur.

Mathematically, these are precision and NPV expressed as:

$$Precision = \frac{TP}{TP + FP} \quad (2.23)$$

$$Negative Predicted Value (NPV) = \frac{TN}{TN + FN} \quad (2.24)$$

Recall

Recall (also known as Sensitivity or the True Positive Rate) quantifies the ability of the model to correctly identify actual positive cases. It reflects the proportion of positive instances that were detected by the classifier. High recall indicates that the model is effective in capturing positive events and produces few false negatives, which is crucial in applications where missing a positive case carries a high cost (*e.g.*, failing to identify a real financial crisis).

In a similar way, the concept of recall can also be applied to the negative class, commonly referred to as Specificity (or the True Negative Rate). While standard recall focuses on the ability of the model to recognize all positive instances, Specificity measures the proportion of actual negative cases that were correctly identified as such. A high specificity indicates that the model is proficient at avoiding false alarms by correctly labeling negative instances, ensuring that the negative class is not misidentified.

Mathematically, these recall and specificity are defined as:

$$Recall (Sensitivity) = \frac{TP}{TP + FN} \quad (2.25)$$

$$Specificity = \frac{TN}{TN + FP}$$

F1-Score

The F1-score is the harmonic mean of precision and recall and provides a single metric that balances both false positives and false negatives. It is particularly useful when precision and recall are equally important or when there is a need to evaluate the model's performance on a specific class using a single metric. A high F1-score indicates that the classifier achieves both high recall and high precision.

Mathematically, the F1-score is defined as:

$$F1 - Score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (2.26)$$

Chapter 3

Illustrative Application

This chapter provides a comprehensive evaluation of the predictive models developed to forecast financial crises, transitioning from automated discovery to refined architectural analysis. Section 3.1 focuses on the initial experimental phase using the Tree-based Pipeline Optimization Tool (TPOT) AutoML framework. Specifically, Subsection 3.1.1 compares three distinct automated searches, comprising two random, distinct AutoML explorations and a final directed output search to identify the most effective genetic programming paths for predicting the "crisis" target variable. These searches differ significantly in their underlying data complexity and feature engineering strategies; while some experiments utilize only the original variables, others incorporate rolling window variables to capture temporal dependencies. Following this, Subsection 3.1.2 delivers a granular performance analysis and confusion matrix for the optimal model identified through these automated iterations.

The subsequent analysis in Section 3.2 represents a shift from the automated discovery of Section 3.1 toward a comparative, methodology-driven phase, evaluating manually constructed and theoretically inspired ML pipelines against those systematically discovered by TPOT. This section is divided into two distinct subsections based on the nature of the feature sets employed. Subsection 3.2.1 evaluates five ML pipelines, including their specific architectures and ensemble structures, using the standard or original variables exclusively. It further assesses model generalization by plotting F1-score trends (training vs. testing) for the highest-performing pipelines. In contrast, Subsection 3.2.2 differentiates itself by shifting the analytical focus toward temporal dynamics; it evaluates ten pipelines specifically optimized for newly created rolling window features. While this subsection also employs F1-score trend analysis for its best-performing pipelines, its primary distinction lies in determining how the integration of windowed data affects predictive stability compared to the original feature approach in Subsection 3.2.1.

3.1 TPOT-Optimized Models

This section initially presents the results of three primary TPOT AutoML experiments conducted to identify the most effective EWS for financial crises. Section 3.1.2 presents the classification performance of the most successful iteration, followed by the confusion matrix and predictive reliability.

3.1.1 Comparative Analysis

We used the TPOT genetic programming framework to explore a population size of 100 random pipelines over 100 generations with 5-fold cross-validation, meaning that each final pipeline is the optimal survivor of approximately 10,000 internal pipeline evaluations. By setting the optimization target to macro-F1, the search was directed to prioritize treating both classes as equally important and a balance between precision and recall. The search is specifically tuned for the requirements of the financial crisis classification task. Table 3.1 shows the experimental results from three distinct AutoML searches conducted through the TPOT framework. Each experiment shared a common goal of predicting the "crisis" target variable, while they significantly differed in data complexity and feature engineering strategies.

Table 3.1: *The experimental results from three distinct AutoML searches conducted through the TPOT framework. Each experiment shared a common goal of predicting the "crisis" target variable, while they significantly differed in data complexity and feature engineering strategies.*

Feature	Experiment 1 (Base)	Experiment 2	Experiment 3
Dataset shape	(342,9)	(342,9)	(342,45)
Covariate count	9 (original variables)	9 (original variables)	45 (inclusion of rolling window variables)
Avg Training CV Score	0.8229	0.8114	0.8287
Best CV Score (F1 Score)	0.7431	0.7445	0.7798
Stacking Layers	4 SGDClassifiers	1 LinearSVC	1 SGDClassifier
Final Estimator	XGBClassifier	XGBClassifier	XGBClassifier
Preprocessing Steps	None	None	Feature Union + RFE
Optimization Duration	0:35:37	0:04:32	1:21:27

A critical finding from the comparison made in Table 3.1 is the significant impact of the temporal feature on model performance. Experiment 3 achieved the best predictive performance with an F1 score of 0.7798, compared to 0.7431 and 0.7445 achieved by the baseline models. Increasing the number of covariates from 9 to 45, including the lagged rolling

window versions for all the macroeconomic variables present in the original dataset, helped the model better capture the temporal dependencies inherent in financial crisis data. However, this improvement came at a substantial computational cost, with the optimization duration extending to over 81 minutes, more than double the time of the standard baseline.

Furthermore, the search process for Experiment 3 converged to an internal cross-validation (CV) score of 0.8202 by the 100th generation, which represents the performance of the model on the training data. There is a slight drop from the training CV score (0.82) to the test score (0.74), suggesting a mild degree of overfitting, which is common in complex automated searches. However, these results remain statistically robust given the balanced dataset.

The architectural complexity of the exported pipelines varied between runs, highlighting the ability of TPOT to adapt to different data shapes. All three experiments converged on XGBClassifier as the final estimator, suggesting its robust suitability for specific datasets. Stacking architecture in Experiment 1 involved using four unique SGDClassifier layers, while Experiment 2 achieved nearly identical results (0.7445) in just around 4 minutes by utilizing a single-layer LinearSVC stacking approach. Experiment 3, the best pipeline, utilized the most sophisticated preprocessing techniques, employing a FeatureUnion of function transformers and Recursive Feature Elimination (RFE), allowing the model to see the original data alongside the transformed data simultaneously. Furthermore, ExtraTreesClassifier is used to manage the high-dimensional 45-feature space before passing data to the final XGBoost model to find a better decision boundary for "Crisis" vs "Non-crisis" events.

3.1.2 Classification Performance and Confusion Matrix

The classification performance analysis of the class-specific metrics and the confusion matrix analysis of the "best pipeline" (Experiment 3) identified by the genetic algorithm in Section 3.1.1 are recorded in Table 3.2 and Table 3.3.

The optimized model with the rolling window scenario achieved an overall accuracy of 0.74 on the test set, with a balanced macro-average F1 score of 0.74. The classification report in Table 3.1 reveals a notable trade-off between the two classes. For non-crisis events (class 0), the model demonstrates a high recall of 0.88, indicating the strong ability to recognize this kind of event. However, the precision remains at 0.69. Conversely, for crisis events (class 1), the model presents high precision (0.83) with low recall (0.60). In other words, about 40% of actual crisis events are missed by the pipeline.

Table 3.2: *Classification performance of the best performed model (Experiment 3) conducted through the TPOT AutoML framework.*

	precision	recall	F1-score	support
0 (No crisis)	0.69	0.88	0.77	58
1 (Crisis)	0.83	0.60	0.70	58
accuracy	—	—	0.74	116
macro avg	0.76	0.74	0.74	116
Weighted avg	0.76	0.74	0.74	116

The confusion matrix analysis presented in Table 3.3 provides a detailed view of the performance of the model, revealing how the XGBClassifier and its preceding SGDClassifier stacking layer handled the binary classification. Out of 58 non-crisis instances (class 0), the model correctly identified 51 instances (true negatives), while 7 (false positives) were incorrectly identified as non-crisis events. However, out of 58 crisis instances (class 1), the model was able to identify only 35 (true positives) instances, leaving 23 (false negatives) instances undetected. This performance highlights that the model’s stability in the prediction of non-crisis events with the help of rolling features, which are effective in filtering out the noise.

Table 3.3: *Confusion matrix analysis for the best performed model (Experiment 3) conducted through the TPOT AutoML framework.*

	Predicted: No Crisis (Class 0)	Predicted: Crisis (Class 1)
Actual: No Crisis (Class 0)	51	7
Actual: Crisis (Class 1)	23	35

3.1.2.1 Predictive Reliability

In this section, we discuss how the model accuracy is on predicting stable periods and crisis events. The optimized rolling window configuration (Experiment 3) presented in Table 3.2 demonstrates a high degree of reliability in its prediction of crisis events with a Precision of 0.83. In other words, there is a probability of 83% that the model recognizes when a crisis is not happening, thus avoiding false alarms. However, the reliability profile also highlights that the model failed to detect 23 actual crisis events with a Recall of 0.60 for the crisis class. This suggests that while the system’s "alerts" are highly trustworthy, the occurrence of a crisis signal is less definitive. Consequently, with high precision, the

system is credible as an early-warning preventative indicator rather than a sole indicator of economic stability.

3.2 Comparative Framework of the Results of the AutoML Pipeline

To evaluate the predictive potential of the proposed approach, a systematic comparative framework was developed. By employing Genetic Programming (GP), utilized in the TPOT pipeline, the framework incorporates different ML algorithms that are not constrained by human bias towards specific algorithms. This framework departs from the traditional "single-algorithm" approach, instead utilizing a complex, multi-layered stacking architecture of ML algorithms to measure incremental performance gains. The comparative framework was tested against two different dataset structures. Section 3.2.1 analyzes the different stacking architectures with standard or original covariates, while Section 3.2.2 discusses the results when the dataset integrates temporal features into different stacking architectures. The core distinction lies in how these rolling window transformations impact the model's ability to capture time-dependent crisis signals compared to the original variables used in the previous Subsection 3.2.1. Furthermore, the comparative framework evaluates these pipelines across two dimensions: structural topology (the arrangement and estimators) and predictive robustness (balance between precision and recall).

3.2.1 Analysis of the Standard Dataset

The evaluation of the first five model architectures provides a critical baseline for understanding how different model architectures interact with the original covariate space. The results summarized in Table 3.4 reveal a clear hierarchy of structural complexity of the models and their performance. Pipe 1, which utilizes a FeatureUnion to process data through parallel paths (a copy of the original data and max normalization) before reaching an ExtraTreesClassifier, emerged as the superior configuration for this dataset. It achieved the highest recorded Precision for class 1 (0.89) and a peak Accuracy of 0.76. This suggests that for the standard dataset, providing the estimator with multiple mathematical "views" of the features is more effective for noise reduction and signal isolation than layering multiple estimators.

In contrast, Pipe 3 and Pipe 5 represent highly complex, multi-layered architectures, with Pipe 5 utilizing a five-layer stack of SGD, BernoulliNB, and XGBoost. Despite this computational depth, Pipe 5 underperformed

Table 3.4: *Summary of the standard data with the evaluated model architectures and ensemble structures.*

Pipeline	Model Architecture	Class	Precision	Recall	F1 Score	Accuracy
Pipe 1	FeatureUnion (Copy + Max Normalization) → ExtraTrees	0	0.69	0.93	0.79	0.76
		1	0.89	0.59	0.71	
Pipe 2	XGBoost	0	0.68	0.88	0.77	0.73
		1	0.83	0.59	0.69	
Pipe 3	ExtraTrees → XGBoost → XGBoost (Stacked)	0	0.69	0.91	0.79	0.75
		1	0.87	0.59	0.70	
Pipe 4	Decision Tree → Bernoulli Naive Bayes (Stacked)	0	0.67	0.81	0.73	0.71
		1	0.76	0.60	0.67	
Pipe 5	SGD → XGBoost → BernoulliNB → SGD → XGBoost (Stacked)	0	0.68	0.84	0.75	0.72
		1	0.80	0.60	0.69	

→ : sequential stacking (Output used as input)

relative to the simpler Pipe 5, yielding a lower F1-score (0.69) and accuracy of 0.72. This phenomenon suggests that without new information from engineered features, the addition of meta-estimators in a deep chain is likely to lead to lower predictive performance.

Furthermore, the atomic baseline model (Pipe 2), represented by the single XGBoost Classifier, serves as an important benchmark for efficiency with an accuracy of 0.73 and Class 1 Precision of 0.83; its performance is remarkably close to the much more complex architectures. The minimal performance difference between the single-learner baseline (Pipe 2) and five-layer model (Pipe 5) suggests that for the original data structure, the primary limiting factor is not the complexity of the model but the representative capacity of the features themselves.

To assess the robustness of the evolved architectures, Figure 3.1 shows the generated learning curves for Pipelines 1, 2, 3, and 5. These plots illustrate the evolution of the F1 score for both training and testing sets. This analysis is critical for detecting potential overfitting and confirming whether the dataset size is sufficient for the complexity of stacking architectures.

Across all four pipelines depicted in Figure 3.1, the F1-score suggests that in the initial phase (sample dimension < 50), the models exhibit a high difference between the two variables. However, as the sample size approaches 100, the gap between training and testing performance narrows significantly. The most immediate and stable convergence is shown by Pipe 1, after reaching a "smooth" convergence of 0.75. It suggests that the FeatureUnion architecture is highly robust to small fluctuations in the data and generalizes exceptionally well. Pipe 2, which is based solely on the XGBoost baseline model, shows frequent oscillations even at higher sample dimensions. Specifically, Pipe 5 displays a wider gener-

alization gap, suggesting that the extreme complexity of the 5-layer stack allows the model only to memorize the training data more effectively than it generalizes to the test set.

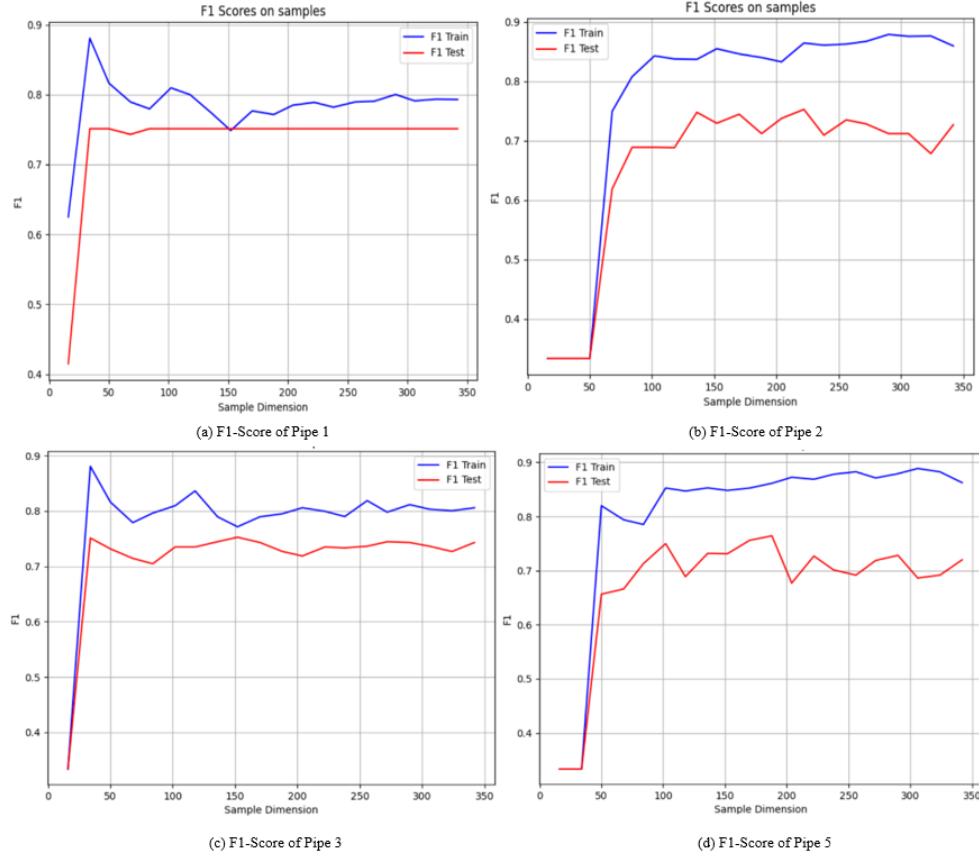


Figure 3.1: The $F1$ -score performance trends for four ML pipelines as a function of sample dimension. Each subplot corresponds to a different pipeline configuration: (a) Pipeline 1, (b) Pipeline 2, (c) Pipeline 3, and (d) Pipeline 5. In every panel, the blue curve represents training $F1$ scores while the red curve shows testing $F1$ scores, allowing direct comparison of model fit and generalization.

3.2.2 The impact of Feature Engineering

This section evaluates ten distinct model architectures, ranging from Pipeline 6 to Pipeline 15, which were developed to leverage advanced temporal dynamics. To enhance the predictive power of these models, the input space was expanded from 9 standard variables to 45 variables through the integration of rolling window features. This transformation is critical as it allows the models to capture latent temporal dependencies and localized trends within the dataset that static variables often overlook. The specific model architectures, along with their respective classification performance metrics, are systematically documented in Ta-

ble 3.5, providing a clear comparison of how these expanded feature sets influence the accuracy and robustness of each pipeline.

The F1-score served as the primary metric for assessing model robustness, particularly given the performance disparity between the two classes. As depicted in Table 3.5, the baseline model implemented in Pipeline 11 struggled with low Class 1 Recall (0.48), where nearly half of the positive instances were misclassified. The impact of feature engineering was also observed in the structural configurations of the pipelines, specifically regarding how original features were passed through the ensemble. Though Pipeline 6 and Pipeline 7 used similar models, the number of features or variables transferred through the models is different. Pipeline 6 utilizes a *FeatureUnion* with *FunctionTransformer(copy)* to ensure the raw features were preserved alongside stacked predictions, while Pipeline 7 is composed of a streamlined sequential structure and achieved an F1-score of 0.72. This suggests that the sequential architecture enables the meta-learner (*ExtraTrees*) to generalize more effectively by leveraging high-dimensional, redundant representations.

Table 3.5: *Summary of the Feature Engineered dataset with the evaluated model architectures and ensemble structures.*

Pipeline	Model Architecture	Class	Precision	Recall	F1 Score	Accuracy
Pipe 6	Gradient Boosting → Linear SVM → SGD → ExtraTrees (Stacked)	0	0.68	0.91	0.78	0.74
		1	0.87	0.57	0.69	
Pipe 7	Gradient Boosting → Linear SVM → SGD → ExtraTrees (Stacked)	0	0.70	0.90	0.79	0.76
		1	0.86	0.62	0.72	
Pipe 8	Linear SVM XGBoost → ExtraTrees	0	0.66	0.91	0.77	0.72
		1	0.86	0.53	0.66	
Pipe 9	Raw features → Binarizer → Decision Tree	0	0.65	0.86	0.74	0.70
		1	0.79	0.53	0.64	
Pipe 10	Binarizer → Decision Tree	0	0.65	0.86	0.74	0.70
		1	0.79	0.53	0.64	
Pipe 11	Decision Tree	0	0.62	0.86	0.72	0.67
		1	0.78	0.48	0.60	
Pipe 12	BernoulliNB → Linear SVM → SGD → ExtraTrees (Stacked)	0	0.68	0.88	0.77	0.73
		1	0.83	0.59	0.69	
Pipe 13	ANOVA FS → RFE → Linear SVM → ExtraTrees (Stacked)	0	0.72	0.91	0.80	0.78
		1	0.88	0.64	0.74	
Pipe 14	RFE → Linear SVM → ExtraTrees (Stacked)	0	0.72	0.91	0.80	0.78
		1	0.88	0.64	0.74	
Pipe 15	ANOVA FS → Linear SVM → ExtraTrees (Stacked)	0	0.68	0.90	0.78	0.74
		1	0.85	0.59	0.69	

→ : sequential stacking (Output used as input)

(stacked) : entire chain is stacking ensemble

|| : parallel branches (FeatureUnion)

FS : feature selection

The introduction of Binarization in Pipelines 9 and 10 marginally improved the metric, but the substantial advancement occurred through the integration of Recursive Feature Elimination (RFE). Pipelines 13 and

14 achieved the highest accuracy (78%) and an F1-score of 0.74 for class 1. By iteratively removing the least significant feature, RFE allowed the stacked ensemble (Linear SVM $\rightarrow$ ExtraTrees) to focus on high-variance predictors, effectively reducing the noise. Notably, the inclusion of ANOVA Feature Selection (ANOVA FS), a univariate filter method that ranks features based on their statistical significance (F-score) to the target variable, was tested in Pipeline 13. However, adding ANOVA FS before RFE yielded results identical to using RFE alone (Pipe 14). Furthermore, the performance drop observed in Pipeline 15 (which lacked RFE) suggests that for this specific dataset, RFE’s wrapper-based approach was sufficient to capture the optimal feature subset, rendering the preliminary statistical filtering of ANOVA redundant.

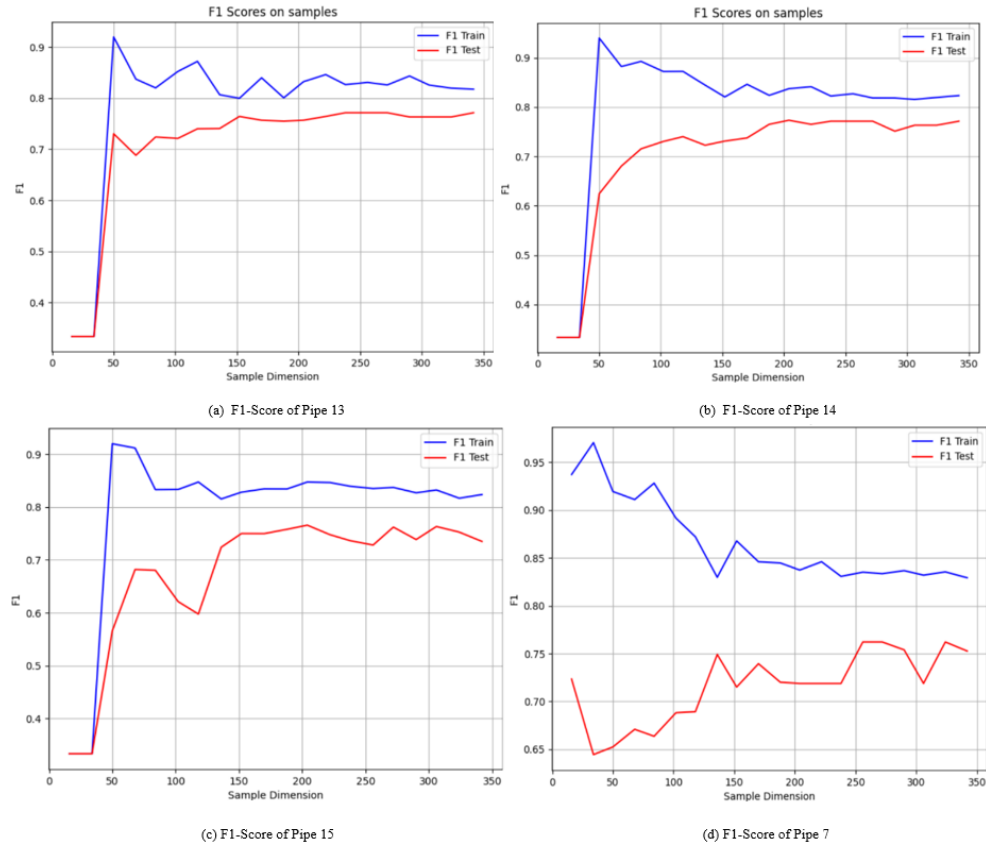


Figure 3.2: The F1-score performance trends for four ML pipelines as a function of sample dimension. Each subplot corresponds to a different pipeline configuration: (a) Pipeline 13, (b) Pipeline 14, (c) Pipeline 15, and (d) Pipeline 7. In every panel, the blue curve represents training F1 scores while the red curve shows testing F1 scores, allowing direct comparison of model fit and generalization.

The learning curves of the best performing Pipelines 13, 14, 15, and 7 are depicted in Figure 3.2, which provides critical insights into the relationship between the sample size and model convergence. A common characteristic across all four plots is the sharp increase in both training

and test F1-score, once the sample dimension exceeds 50 observations.

In Pipelines 13 and 14 (Figure 3.1), the learning curves demonstrate high stability around 0.76-0.78 and strong generalization. This behavior suggests that the RFE effectively mitigated the overfitting by removing redundant features, allowing the model to perform better as the sample dimension increases. The relatively flat trajectory of the test curve at higher sample dimensions indicates that these pipelines have reached a point of stability where additional data yields diminishing returns in F1-score. In contrast, the F1-score of Pipeline 15 (ANOVA-based) exhibits significant fluctuations between 100 and 150 samples before stabilizing. This may be due to the filter-based nature of the ANOVA failing to capture complex feature interactions. Unlike other models, Pipeline 7 maintained a higher training F1-score (0.82) even at a large sample size, but with a gap relative to the test score. Though the sequential architecture in Pipeline possesses a high predictive power, there exist fluctuations in specific sample variations.

Chapter 4

Discussion

The experimental results across the original and feature-engineered data reveal a fundamental "feature-complexity paradox" in financial crisis prediction. The primary objective was to determine if automated architectural discovery could outperform human-defined baseline models in predicting future crises on the basis of macro-economic and monetary indicators. This section begins with an analysis of the architectural complexity of the best-performing models identified in Chapter 3, followed by an assessment of the impact of temporal feature engineering. Furthermore, we discuss the classification performance and reliability of these models, along with the convergence and stability of the learning curve analysis of human-defined baseline models.

Table 4.1 summarizes the best-performing models categorized by their experimental context (original vs. feature-engineered datasets). These selections are prioritized based on a balance of accuracy, F1-Score, and Class 1 (crisis) precision. In the analysis of the original dataset (Section 3.2.1), a key distinction emerges between the structural orientations of the models. Horizontal architectures (such as FeatureUnion) prioritize parallel data processing to capture diverse perspectives of the same input, whereas vertical architectures (stacking) focus on hierarchical abstraction through sequential layers. The superior performance of Pipe 1 (FeatureUnion) over the more complex five-layer stack, like Pipe 5, suggests that with the low-dimensional data, "horizontal" complexity is more efficient than "vertical" complexity. Specifically, Pipe 1's horizontal approach processes raw data through parallel paths, providing multiple simultaneous views, such as comparing raw data against normalized variants, which proves more effective than increasing the vertical depth of a stacking architecture without the injection of new feature information. This implies that the multiple stacked meta-estimators often result in the memorization of historical noise rather than uncovering underlying patterns. This is also visually confirmed by the learning curves in Figure 3.1, where Pipe 1 demonstrates a tighter generalization gap compared to the

oscillations of Pipe 5.

The transition to the feature-engineered dataset proved that feature quality, rather than algorithm complexity, is the primary drive of predictive power. Expanding the dataset from 9 to 45 variables allowed not only the TPOT pipeline but also the human-defined pipeline to capture the temporal dependencies inherent in financial data. The inclusion of lagged rolling window features allowed the corresponding TPOT-optimized pipeline (Experiment 3) to achieve a macro-F1 score of 0.74 and Pipeline 14 to achieve a score of 0.77. The addition of 36 extra features allowed the model to account for the historical evolution and lagged dependencies of the variables. Recursive Feature Elimination (RFE) iteratively removed the least significant features, allowing the model to focus on the high-variance predictors. It is evident from the learning curves in Figure 3.2 that Pipeline 14 reached a higher stability and strong generalization with fewer oscillations compared to Pipeline 13 with ANOVA filters, confirming that the wrapper-based selection is better suited for capturing the non-linear interactions that typically exist in financial risk tasks.

Table 4.1: *Comparison of the top-performing pipelines identified in Section 3: the standard dataset, the feature-engineered dataset, and the final TPOT optimization search. For each setting, the best pipeline is reported along with its model architecture and key evaluation metrics, including Class 1 precision, Class 1 recall, macro-F1, and overall accuracy.*

Experimental Context	Best Pipeline	Model Architecture	Class 1 Precision	Class 1 Recall	F1-Score (Macro)	Accuracy
Standard Dataset	Pipe 1	FeatureUnion (Copy + Max Normalization) → ExtraTrees	0.89	0.59	0.75	0.76
Feature Engineered	Pipe 14	RFE → Linear SVM → ExtraTrees (Stacked)	0.88	0.64	0.77	0.78
Final TPOT Search	Experiment 3	RFE + XGBoost (Final Estimator)	0.83	0.60	0.74	0.74

The classification performance of the final TPOT-optimized model (Experiment 3) reveals a critical asymmetry between the precision and recall. As shown in Table 4.1, the model achieved a high precision of 0.83 but a more modest recall of 0.60 for the crisis class. Furthermore, from the confusion matrix shown in the Table 3.3, the discrepancy between the true negative rate (87.9%) and the true positive rate (69.3%) highlights a critical area for error analysis. Despite containing 45 distinct covariates, the lagged values model did not exhibit required crisis alerts in the presence of true crisis events. This may be due to the nature of XGBClassifier, which utilizes the model functions as an additive ensemble, prioritizing individual feature contributions over complex multi-variable interactions.

While this helps to mitigate the risk of overfitting on the sample size of 342 observations, the model may overlook crisis triggers that may occur through non-linear feature interactions. In a policy-making context, it ensures that when the system triggers an alert, there is an 83% probability of an actual crisis forthcoming, helping to avoid false alarms that could lead to expensive and unnecessary market interventions. However, 40% of crisis events remain undetectable, likely due to events such as geopolitical shocks that are not captured in standard macroeconomic datasets.

The primary takeaway is that Pipeline 14 represents the absolute maximum predictive capacity out of all the models that are discussed in this study. However, Experiment 3 represents the survivor of a much broader, 10,000-pipeline genetic search that prioritized a macro-F1 fitness function.

Chapter 5

Conclusion

This thesis is set out to evaluate the efficiency of Automated Machine Learning (AutoML), specifically the TPOT framework, which uses genetic programming to explore thousands of possible pipelines. TPOT works by evolving combinations of preprocessing steps, feature selectors, and classification algorithms, eventually converging on an "optimal" pipeline that maximizes predictive accuracy for the specific dataset provided.

The initial phase of the study focused on transforming a raw longitudinal dataset into a structured format suitable for crisis prediction. The cleaning procedure targeted 129 countries from 1983 to 2017, narrowing the scope to nine core economic indicators, such as ISO country code (*isocode*), year, binary target variable if crisis occurred denoted as 1, 0 otherwise (*crisis0*), one-year lagged target variable (*crisis0_1*), real GDP growth rate, logarithm of capita GDP, inflation, real interest rate, broad money(M2) over foreign exchange, growth rate of private credit and growth rate of private credit. To maintain analytical consistency, a auxiliary dummy variables *aux1* and *aux2* are created to flag missing values present in the dataset. By adding missing country-year rows and assuming a non-crisis state (0) for those periods, the dataset was standardized to 4,515 observations across 15 variables. To move beyond static snapshots and capture the temporal dependencies inherent in financial cycles, a rolling window feature engineering strategy was implemented. This involved a five-year backward-looking window for eight key variables, expanding the feature space from 9 standard variables to 45 temporal variables. For country-year observations with insufficient history, the earliest available record was repeated to ensure a complete, lag-based feature set without data loss. This transformation allowed the subsequent models to detect volatility and trends rather than just isolated annual fluctuations.

A primary challenge of this research was the construction of a balanced dataset, as financial crises are statistically rare compared to periods of

stability. To address this, the study introduced a metadata-driven stratified sampling strategy. This began with Geographic Aggregation, grouping countries into five continental regions (merging North America and Oceania due to data sparsity). These groups were further refined through variable binning of land area and GDP per capita, resulting in a composite stratification identifier (*sel*). This identifier allowed for a precise, balanced under-sampling procedure. By matching the frequency of the minority crisis class within these specific economic and geographic strata, the study produced a perfectly balanced Training Set (342 observations) and Testing Set (116 observations). Any observations from groups lacking class diversity were moved to an external validation set (test3). This rigorous 70/30 split ensured that the model remained sensitive to crisis patterns across diverse regional scales, providing a robust foundation for the AutoML and architectural comparisons that followed.

The evaluation of these models was conducted in two distinct phases to isolate the impact of both architecture and feature engineering:

- **Automated Discovery:** The initial results focused on the TPOT AutoML algorithm, which automatically generated optimized pipelines. This phase served as a benchmark, testing the framework’s ability to handle both original variables and newly engineered features without manual intervention.
- **Architectural Refinement:** The second phase systematically documented the performance of specific machine learning pipelines inspired by the TPOT findings. This was further divided into:
 - Standard Analysis (Pipelines 1–5): An evaluation of different model architectures and ensemble structures using only the original variables.
 - Temporal Analysis (Pipelines 6–15): An exploration of more complex architectures, including stacked ensembles leveraging rolling window variables to capture the dynamic dependencies inherent in economic time-series data.

The experimental results reveal a notable consideration in the application of Evolutionary AutoML to financial crisis prediction: while the TPOT algorithm successfully explored a vast search space of over 10,000 pipelines, the highest predictive performance was ultimately achieved by a human-refined model (Pipeline 14) with a Macro-F1 score of 0.77 that utilized similar model architecture to best-performed TPOT-optimized model, which achieved a Macro F1-score of 0.74. This discrepancy highlights the important issue in applying AutoML to financial data, commonly referred to as the "optimization paradox," whereby increased optimization does not necessarily improve performance. Although TPOT reported a superior internal cross-validation score of 0.82, its performance

on unseen data declines, suggesting that the genetic programming process overfitted the specific training folds.

The best-performing human-refined pipeline (Pipe 14) was architecturally inspired by TPOT’s selection of Recursive Feature Elimination (RFE) and stacking estimators. By substituting the high-variance XGBClassifier with ExtraTreesClassifier and tuning with LinearSVC stacking layer, the human-defined model better managed the noise that is inherent in the financial crisis indicators. The 3% gain in F1-score achieved through human intervention represents the recovery of generalization loss, proving that human oversight is required to steer AutoML away from high-complexity, low-robustness solutions. Consequently, this study proposes that the optimal workflow for financial distress modeling is not fully autonomous but rather a hybrid approach where AutoML provides the architectural blueprint and human domain knowledge provides the final refinement.

The performance analysis of Pipeline 14 highlights a strong ability to identify non-crisis events (Class 0), achieving a precision of 0.72 and a high recall of 0.91. For crisis events (Class 1), the pipeline achieved a notably high precision of 0.88, meaning that nearly 9 out of 10 crisis alerts were accurate, though the recall was 0.64, reflecting that some crises were not detected, likely because unexpected factors, such as geopolitical shocks, are not captured in typical macroeconomic indicators, highlighting the limits of data-driven early-warning systems. In practice, this performance is valuable for macro-prudential policy, where regulatory measures rely on reliable crisis alerts to maintain the stability of the financial system and to prevent risks. By reducing the false alarms, Pipe 14 allows policymakers to take preventative actions before a crisis destabilizes the financial system.

A significant limitation encountered in developing the Early Warning System (EWS) for financial crisis was the availability and quality of data. Several macroeconomic indicators contained a substantial proportion of missing values, alongside a complete lack of observations for certain countries in specific years, which can reduce the reliability of the predictive models. Although necessary preprocessing efforts, including imputation and feature selection techniques, were adapted to address this challenge, statistically, these efforts might introduce synthetic noise into the model. In the context of this study, such data limitations likely contributed to the performance observed in the TPOT generations and Pipe 14. This highlights the importance of continuous updating, and the expansion of the datasets to include consistent indicators across countries would improve the EWS’s effectiveness in detecting financial crises.

A fundamental challenge in crisis modeling is the rarity of the target event. Financial crises or systemic bankruptcies in a country are infrequent relative to periods of financial stability. This natural class imbal-

ance typically biases predictive models towards the majority class (non-crisis). This study addressed this limitation through a metadata-driven stratified sampling strategy. By incorporating country-level information such as continent, land area, and GDP per capita and combining it with binning techniques to mitigate the impact of class imbalance, it ensures that both crisis and non-crisis observations are adequately represented. This approach ensures that the model learns from a diverse set of economic profiles rather than being overwhelmed by the majority class.

The integration of the Tree-based Pipeline Optimization Tool (TPOT) significantly enhanced the system’s predictive power through automated model selection and hyperparameter optimization, allowing exploration of complex model architectures such as Pipe 14. However, this optimization introduces the "black-box" element to the pipeline, limiting interpretability and making it less transparent. Mathematical robustness and the complexity of the evolved pipelines obscure the reasoning behind specific crisis alerts. In a policy-making framework where trust and accountability are paramount, this poses a challenge. The high precision of Pipe 14 demonstrates reliable alerts, but decision-makers must be cautious when relying on predictions from automated pipelines.

Finally, the temporal and geographic generalizability of the results remains a critical consideration. Economic landscapes are dynamic; predictive patterns established during a certain period might not remain valid in a digital service-dominant economy. Because economic environments vary significantly across continents, the patterns identified by the model in the current dataset may not be universally applicable to the future. Stratified sampling strategy and balanced dataset construction improve the representativeness; caution is required when applying the model to a new context. Therefore, the EWS should be viewed as a tool that requires continuous recalibration to account for shifting global economic structures and the events that fall outside the historical training distribution.

The findings of this study suggest that the future of financial crisis modeling lies not only in the replacement of the human analyst but also in a Human-in-the-Loop (HITL) AutoML framework. TPOT serves as a highly efficient, innovative engine that can propose complex feature-engineering paths, while the human researcher serves as the strategic regularizer, ensuring the model remains grounded in economic reality.

Bibliography

- [1] Lucia Alessi and Carsten Detken. “Identifying excessive credit growth and leverage”. In: *Journal of financial stability* 35 (2018), pp. 215–225.
- [2] Jan Babecký et al. *Banking, debt and currency crises: early warning indicators*. Tech. rep. ECB Working Paper, 2012.
- [3] Wolfgang Banzhaf et al. *Genetic programming: an introduction: on the automatic evolution*. Morgan Kaufmann Publishers Inc., 1998.
- [4] James Bergstra and Yoshua Bengio. “Random search for hyperparameter optimization”. In: *The journal of machine learning research* 13.1 (2012), pp. 281–305.
- [5] Claudio Borio and Mathias Drehmann. “Towards an operational framework for financial stability: "fuzzy" measurement and its consequences”. In: *Documentos de Trabajo (Banco Central de Chile)* 544 (2009), p. 1.
- [6] Leo Breiman. “Bagging predictors”. In: *Machine learning* 24.2 (1996), pp. 123–140.
- [7] Leo Breiman. “Random forests”. In: *Machine learning* 45.1 (2001), pp. 5–32.
- [8] Eric Brochu, Vlad M Cora, and Nando De Freitas. “A tutorial on Bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning”. In: *arXiv preprint arXiv:1012.2599* (2010).
- [9] Marcus Buckmann, Andreas Joseph, and Helena Robertson. “An interpretable machine learning workflow with an application to economic forecasting”. In: *International Journal of Central Banking* 19.4 (2023), pp. 449–552.
- [10] Peter Bühlmann and Bin Yu. “Analyzing bagging”. In: *The annals of Statistics* 30.4 (2002), pp. 927–961.
- [11] Matthieu Bussiere and Marcel Fratzscher. “Towards a new early warning system of financial crises”. In: *journal of International Money and Finance* 25.6 (2006), pp. 953–973.

- [12] Giovanni Caggiano et al. “Comparing logit-based early warning systems: Does the duration of systemic banking crises matter?” In: *Journal of Empirical finance* 37 (2016), pp. 104–116.
- [13] Bo Chen et al. “A comprehensive survey on automated machine learning for recommendations”. In: *ACM Transactions on Recommender Systems* 2.2 (2024), pp. 1–38.
- [14] Vinod Chugani. *What is Boosting in Machine Learning?* Accessed: 2026-02-15. DataCamp. Aug. 16, 2024. URL: <https://www.datacamp.com/tutorial/what-is-boosting>.
- [15] Corinna Cortes and Vladimir Vapnik. “Support-vector networks”. In: *Machine learning* 20.3 (1995), pp. 273–297.
- [16] E Philip Davis and Dilruba Karim. “Comparing early warning systems for banking crises”. In: *Journal of Financial stability* 4.2 (2008), pp. 89–120.
- [17] Asli Demirgüç-Kunt and Enrica Detragiache. “Cross-country empirical studies of systemic bank distress: a survey”. In: *National institute economic review* 192 (2005), pp. 68–83.
- [18] Matthias Feurer et al. “Efficient and robust automated machine learning”. In: *Advances in neural information processing systems* 28 (2015).
- [19] David Firth. “Bias reduction of maximum likelihood estimates”. In: *Biometrika* 80.1 (1993), pp. 27–38.
- [20] Yoav Freund and Robert E Schapire. “A decision-theoretic generalization of on-line learning and an application to boosting”. In: *Journal of computer and system sciences* 55.1 (1997), pp. 119–139.
- [21] Yoav Freund et al. “An efficient boosting algorithm for combining preferences”. In: *Journal of machine learning research* 4.Nov (2003), pp. 933–969.
- [22] Jerome Friedman, Trevor Hastie, and Robert Tibshirani. “Additive logistic regression: a statistical view of boosting (with discussion and a rejoinder by the authors)”. In: *The annals of statistics* 28.2 (2000), pp. 337–407.
- [23] Jerome H Friedman. “Greedy function approximation: a gradient boosting machine”. In: *Annals of statistics* (2001), pp. 1189–1232.
- [24] A.G. Ganie and S. Dadvandipour. “From big data to smart data: a sample gradient descent approach for machine learning”. In: *Journal of Big Data* 10 (2023), p. 162.
- [25] GeeksforGeeks. *Probabilistic Models in Machine Learning*. Last updated: 23 July 2025; Accessed: 2026-02-15. GeeksforGeeks. July 23, 2025. URL: <https://www.geeksforgeeks.org/machine-learning/g/probabilistic-models-in-machine-learning/>.

- [26] GeeksforGeeks. *Tree Based Machine Learning Algorithms*. Last updated: 23 July 2025. 2025. URL: <https://www.geeksforgeeks.org/machine-learning/tree-based-machine-learning-algorithms/>.
- [27] Pierre Geurts, Damien Ernst, and Louis Wehenkel. “Extremely randomized trees”. In: *Machine learning* 63.1 (2006), pp. 3–42.
- [28] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *An introduction to statistical learning*. Springer, 2009.
- [29] Georg Heinze and Michael Schemper. “A solution to the problem of separation in logistic regression”. In: *Statistics in medicine* 21.16 (2002), pp. 2409–2419.
- [30] Dominik Herrmann, Rolf Wendolsky, and Hannes Federrath. “Website fingerprinting: attacking popular privacy enhancing technologies with the multinomial naive-bayes classifier”. In: *Proceedings of the 2009 ACM workshop on Cloud computing security*. 2009, pp. 31–42.
- [31] Markus Holopainen and Peter Sarlin. “Toward robust early-warning models: A horse race, ensembles and model uncertainty”. In: *Quantitative Finance* 17.12 (2017), pp. 1933–1963.
- [32] Dong Huang, Jianhuang Lai, and Chang-Dong Wang. “Ensemble clustering using factor graph”. In: *Pattern Recognition* 50 (2016), pp. 131–142.
- [33] Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren. *Automated machine learning: methods, systems, challenges*. Springer Nature, 2019.
- [34] Kehinde Damilola Ilesanmi and Devi Datt Tewari. “An Early Warning Signal (EWS) Model for Predicting Financial Crisis in Emerging African Economies”. In: *International Journal of Financial Research* 12.1 (2020), p. 101.
- [35] Gareth James et al. *An introduction to statistical learning: with applications in R*. Vol. 103. Springer, 2013.
- [36] Òscar Jordà, Moritz Schularick, and Alan M Taylor. “Leveraged bubbles”. In: *Journal of Monetary Economics* 76 (2015), S1–S20.
- [37] Andreas Joseph. “Parametric inference with universal function approximators”. In: *arXiv preprint arXiv:1903.04209* (2019).
- [38] James Max Kanter and Kalyan Veeramachaneni. “Deep feature synthesis: Towards automating data science endeavors”. In: *2015 IEEE international conference on data science and advanced analytics (DSAA)*. IEEE. 2015, pp. 1–10.
- [39] Gary King and Langche Zeng. “Logistic regression in rare events data”. In: *Political analysis* 9.2 (2001), pp. 137–163.

- [40] Ioannis Kosmidis and David Firth. “Bias reduction in exponential family nonlinear models”. In: *Biometrika* 96.4 (2009), pp. 793–804.
- [41] Mr Luc Laeven and Mr Fabian Valencia. *Systemic banking crises revisited*. International Monetary Fund, 2018.
- [42] Tony Lancaster. “The incidental parameter problem since 1948”. In: *Journal of econometrics* 95.2 (2000), pp. 391–413.
- [43] Larhman. *Maximum-margin hyperplane and margin for an SVM*. Wikimedia Commons, image. Licensed under CC BY-SA 4.0. 2018. URL: https://commons.wikimedia.org/wiki/File:SVM_margin.png.
- [44] Trang T Le, Weixuan Fu, and Jason H Moore. “Scaling tree-based automated machine learning to biomedical big data with a feature set selector”. In: *Bioinformatics* 36.1 (2020), pp. 250–256.
- [45] Kevin P Murphy. *Machine learning: a probabilistic perspective*. MIT press, 2012.
- [46] Derrick Mwiti. *Linear Machine Learning Algorithms: An Overview*. Accessed: 2026-02-15. KDnuggets. July 1, 2022. URL: <https://www.kdnuggets.com/2022/07/linear-machine-learning-algorithms-overview.html>.
- [47] Alexey Natekin and Alois Knoll. “Gradient boosting machines, a tutorial”. In: *Frontiers in neurorobotics* 7 (2013), p. 63623.
- [48] Jerzy Neyman and Elizabeth L Scott. “Consistent estimates based on partially consistent observations”. In: *Econometrica: journal of the Econometric Society* (1948), pp. 1–32.
- [49] Mikhail V Oet, John M Dooley, and Stephen J Ong. “The financial stress index: Identification of systemic risk conditions”. In: *Risks* 3.3 (2015), pp. 420–444.
- [50] Randal S Olson and Jason H Moore. “TPOT: A tree-based pipeline optimization tool for automating machine learning”. In: *Workshop on automatic machine learning*. PMLR. 2016, pp. 66–74.
- [51] Randal S Olson et al. “Evaluation of a tree-based pipeline optimization tool for automating data science”. In: *Proceedings of the genetic and evolutionary computation conference 2016*. 2016, pp. 485–492.
- [52] Claudia Pigini. “Penalized maximum likelihood estimation of logit-based early warning systems”. In: *International Journal of Forecasting* 37.3 (2021), pp. 1156–1172.
- [53] Kurtis Pykes. *Gradient Descent*. Towards Data Science, Aug. 13, 2020. 2020. URL: <https://towardsdatascience.com/gradient-descent-811efcc9f1d5/> (visited on 02/11/2026).
- [54] Dario Radecic. *Machine Learning Automation with TPOT*. Packt Publishing Ltd, 2021.

- [55] Carmen Reinhart, Morris Goldstein, and Graciela Kaminsky. “Methodology for an early warning system: The signals approach”. In: (2000).
- [56] Carmen M Reinhart and Kenneth S Rogoff. “The aftermath of financial crises”. In: *American Economic Review* 99.2 (2009), pp. 466–472.
- [57] Pedro Henrique Ribeiro et al. “Benchmarking AutoML algorithms on a collection of synthetic classification problems”. In: *arXiv preprint arXiv:2212.02704* (2022).
- [58] Fabien Scalzo, Robert Hamilton, and Xiao Hu. “Real-time analysis of intracranial pressure waveform morphology”. In: *Adv Top Neurol Disord InTech* (2012), pp. 99–128.
- [59] scikit-learn developers. *Related Projects — scikit-learn Documentation*. Accessed: 2026-02-15. 2025. URL: https://scikit-learn.org/stable/related_projects.html.
- [60] Radhey Shyam and Riya Chakraborty. “Machine learning and its dominant paradigms”. In: *Journal of Advancements in Robotics* 8.2 (2021), pp. 1–10.
- [61] Padhraic Smyth and David Wolpert. “Stacked density estimation”. In: *Advances in neural information processing systems* 10 (1997).
- [62] Erik Strumbelj and Igor Kononenko. “An efficient explanation of individual classifications using game theory”. In: *The Journal of Machine Learning Research* 11 (2010), pp. 1–18.
- [63] Katsuyuki Tanaka, Takuji Kinkyo, and Shigeyuki Hamori. “Random forests-based early warning system for bank failures”. In: *Economics Letters* 148 (2016), pp. 118–121.
- [64] Yingjie Tian, Yuqi Zhang, and Haibin Zhang. “Recent Advances in Stochastic Gradient Descent in Deep Learning”. In: *Mathematics* 11.3 (2023), p. 682.
- [65] Parashuram S Vadar, Tejashree T Moharekar, and Urmila R Pol. “Comparative analysis of automated machine learning libraries: py-caret, h2o, tpot, auto-sklearn, and flaml”. In: *Int. J. Sci. Res. Eng. Manag* 8.11 (2024), pp. 1–8.
- [66] Wikipedia contributors. *XGBoost*. Accessed: 2026-02-11. 2025. URL: <https://en.wikipedia.org/wiki/XGBoost>.
- [67] Ersan YAZAN and M. Fatih Talu. “Comparison of the stochastic gradient descent based optimization techniques”. In: *2017 International Artificial Intelligence and Data Processing Symposium (IDAP)*. 2017, pp. 1–5. DOI: 10.1109/IDAP.2017.8090299.
- [68] Hao Helen Zhang et al. “Variable selection and model building via likelihood basis pursuit”. In: *Journal of the American statistical Association* 99.467 (2004), pp. 659–672.

- [69] Marc-André Zöller, Tien-Dung Nguyen, and Marco F Huber. “Incremental search space construction for machine learning pipeline synthesis”. In: *International Symposium on Intelligent Data Analysis*. Springer. 2021, pp. 103–115.
- [70] Jason Zutty et al. “Multiple objective vector-based genetic programming using human-derived primitives”. In: *Proceedings of the 2015 Annual Conference on Genetic and Evolutionary*. 2015, pp. 1127–1134.